

MODULE 01

1.GENERATIVE AI LANDSCAPE & RESPONSIBLE USE

1.1 Introduction and Module Overview

Generative Artificial Intelligence (GenAI) represents a major shift in the field of computing, where machines are no longer limited to classification or prediction tasks but are capable of creating original text, images, code, and multimedia content. This Module introduces students to the overall landscape of AI systems, maps the progression from traditional machine learning to modern large language models (LLMs), and establishes the necessity of responsible technology use as a foundational principle.

The purpose of this Module is to give learners a conceptual grounding before engaging in practical development activities. Understanding what GenAI can and cannot do is essential for safe deployment and effective academic or professional application. This Module also examines the ethical risks involved with generative systems and introduces globally recognized frameworks for responsible AI usage.

1.2 Definition of Artificial Intelligence, Machine Learning, NLP and Generative AI Artificial Intelligence (AI)

Artificial Intelligence is the broad discipline concerned with developing machines capable of demonstrating behaviors traditionally associated with human intelligence, including learning, problem-solving, reasoning, perception, and language understanding.

AI systems operate by:

- a) Collecting data
- b) Learning patterns or rules
- c) Applying those patterns to perform tasks autonomously

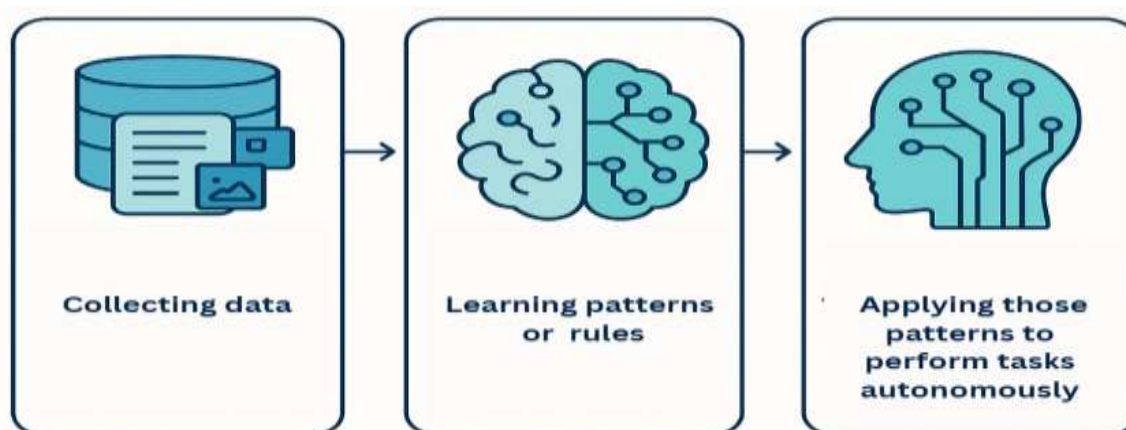


Figure 1.1 : Three-Stage Operation of Artificial Intelligence Systems

Machine Learning (ML)

Machine Learning is a subfield of AI focused on using algorithms to allow machines to learn from data rather than relying on explicit programming. In ML, models analyze large datasets and detect patterns used for predictions or classifications.

Typical examples include:



Email spam filtering



Credit risk classification



Face recognition



Price prediction systems

Table 1.1 : Applications of Machine Learning

S.No	Application Area	Example	How Machine Learning is Used
1	Communication & Email Systems	Email Spam Filtering	ML models analyze email content, sender history, and keywords to classify messages as <i>spam</i> or <i>legitimate</i> .
2	Banking & Finance	Credit Risk Classification	ML evaluates customer income, spending behavior, credit history, and repayments to determine a borrower's default risk.
3	Security & Biometrics	Face Recognition Systems	ML algorithms learn facial features from images to identify or verify individuals for authentication and surveillance.
4	Business & Economics	Price Prediction Systems	ML models study historical prices, demand trends, seasonality, and market indicators to forecast future product or stock prices.

Natural Language Processing (NLP)

Natural Language Processing is the branch of AI dedicated to enabling machines to process, analyze, and understand human language.

Traditional NLP systems perform tasks such as:

- Sentiment analysis
- Named entity recognition
- Text classification
- Machine translation

These systems rely on feature engineering and statistical classification rather than free text generation.

Table 1.2 : Traditional NLP Tasks and Examples

S.No	NLP Task	Meaning	Example
1	Sentiment Analysis	Identifies whether a text expresses positive, negative, or neutral feelings.	A restaurant review saying "Food was excellent" is marked Positive ; "Service was slow" is marked Negative .

2	Named Entity Recognition (NER)	Finds and labels important names such as people, places, organizations, or dates in a text.	In the sentence “ <i>Ravi lives in Chennai and works at TCS</i> ”, Ravi (Person) , Chennai (Location) , and TCS (Organization) are recognized.
3	Text Classification	Groups texts into predefined categories based on their topic or purpose.	An email is classified as Spam or Inbox ; news articles are grouped into Sports , Politics , or Technology .
4	Machine Translation	Converts text from one language to another automatically.	Translating “ <i>Good Morning</i> ” from English to French as “ <i>Bonjour</i> ”.

Generative Artificial Intelligence (GenAI)

Generative AI extends NLP by creating new content, not merely labeling or categorizing existing material. These systems use deep neural architectures trained on enormous text corpora to learn grammar, semantic relationships, and contextual meaning across languages.

Common GenAI tasks include:

- a) Essay writing
- b) Conversational chatbots
- c) Document summarization
- d) Code generation

GenAI systems operate probabilistically, predicting the most semantically appropriate next word (token) during response generation.



Figure 1.2: AI Taxonomy Pyramid

1.3 Evolution of Language Models

The development of AI language processing has progressed through several key stages:

Rule-Based Systems

Rule-Based Systems were the earliest form of computer chat and language programs. These systems worked using **fixed rules written by humans**. They did **not learn or think** on their own. Instead, they matched user sentences with **pre-written patterns** and gave **pre-set replies**. They could respond only to questions they were programmed for and **could not understand meaning or context** like modern AI systems.

Table 1.3 : Rule-Based Systems with Examples

S.No	Concept	Meaning	Example
1	Rule Matching	The computer looks for keyword patterns and follows fixed rules to reply.	If you type “Hello” , the system replies “Hi! How can I help?” because that rule is already written.
2	Pre-written Responses	All responses are stored in advance; the system does not create new answers.	Typing “How are you?” always gives the same reply like “I am fine.”
3	No Learning Ability	Cannot improve or learn from new questions.	Asking a new question like “Explain AI” gives no answer if it was not programmed earlier.
4	Limited Understanding	The system does not understand meaning; it only matches words.	If you write “I am sad today” , it may reply “Tell me more about your family” even if the answer does not fit.

Example:

ELIZA chatbot

ELIZA was an early chatbot that followed this rule-based method.

Table 1.4 : How ELIZA worked

User Says	ELIZA Responds	Why
“I feel tired.”	“Why do you feel tired?”	The word “feel” matches a rule that triggers a question.
“My exams are stressful.”	“Tell me more about your exams.”	The word “exams” activates a stored response pattern.
“What is AI?”	<i>(No useful reply)</i>	No rule exists for that question.

Rule-Based Systems are like **talking machines that follow a fixed instruction book**. They do not understand meanings and cannot learn new things; they reply only based on pre-written rules and patterns. Examples include early chatbots such as **ELIZA**.



Figure 1.3: ELIZA Chatbot

Statistical NLP Models

Statistical NLP models improved over rule-based systems by using **mathematics and probability** to analyze language instead of fixed rules. These models **learn patterns from large text data** and use statistics to predict words, grammar structure, or sentiment.

Common techniques include:

- **N-grams** — predict the next word based on previous words.
- **Hidden Markov Models (HMM)** — identify sequences such as parts of speech or speech patterns.
- **Support Vector Machine (SVM) classifiers** — classify texts into categories such as positive/negative sentiment or spam/not spam.

Table 1.5 : Statistical NLP Models

S.No	Model / Method	Meaning	Example
1	N-grams	Predict the next word using word patterns.	“I am going to” → predicts “school” or “home” based on past data.
2	Hidden Markov Models (HMM)	Recognize word sequences and grammar patterns.	Identifies noun and verb in the sentence: “Ravi <i>plays</i> football.”
3	SVM Classifiers	Sorts or labels text automatically.	Classifies reviews as Positive or Negative .

Table 1.6 : Limitations of Statistical NLP Models

Limitation	Explanation	Example
Required Feature Engineering	Humans must manually decide what parts of text should be used for the model to learn.	Engineers must specify using word counts, sentence length, or keywords before training the system.
Poor Long-Context Understanding	Difficult for models to remember meaning across long sentences or paragraphs.	In long stories, the model forgets who “he” or “she” refers to.

Statistical NLP models use math and probability to understand and classify language better than rule-based systems, but they still require human help to design features and struggle to understand context across long texts.

Neural Language Models

Neural Language Models use deep learning techniques to understand and process human language. The introduction of Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) models improved how computers handle sentences because they process words in sequence and remember previous words while predicting the next ones.

Table 1.7 : Neural Language Models

S.No	Model Type	Meaning	Example
1	RNN (Recurrent Neural Network)	Reads one word at a time and uses earlier words to understand the next word.	In the sentence “ <i>I am going to school</i> ”, the model remembers “I am going to” before predicting “school.”
2	LSTM (Long Short-Term Memory)	An improved RNN that remembers information for a longer time.	In “ <i>Ravi, who lives in Chennai, is my friend</i> ,” the model remembers that Ravi is the subject even after several words.

Table 1.8 : Advantages and Drawbacks

Aspect	Explanation	Example
Advantage – Better Sentence Coherence	Produces more natural and meaningful sentences by remembering word order.	Writes a complete, logical sentence instead of broken phrases.
Drawback – Difficulty with Very Long Text	Still loses focus when processing very long paragraphs or documents.	For long stories, the model may forget early details or mix up subjects.

Neural Language Models improved text understanding by remembering word sequences and building better sentences, but they still struggled to keep track of meaning across very long texts.

Transformers & Large Language Models

The Transformer architecture transformed Natural Language Processing by introducing the method of self-attention.

Instead of reading words one by one like older models (RNN, LSTM), Transformers look at all words in a sentence at the same time and understand how each word relates to every other word. This approach allows models to understand meaning, grammar, and context more effectively, especially in long passages.

Table 1.9 : How Transformers Work

Concept	Meaning	Example
Self-Attention	The model checks how every word is connected to other words in a sentence.	In “ <i>Ravi gave his book to Kumar</i> ”, the model understands that “ his ” refers to Ravi , not Kumar.
Parallel Processing	All words are processed at the same time, not one by one.	A full sentence is evaluated together instead of reading word after word slowly.
Context Understanding	Keeps track of meanings across long text.	In a long paragraph, the model remembers who is being discussed throughout the text.

Modern Large Language Models (LLMs)

Popular Transformer-based systems such as GPT-style models use advanced training techniques to generate accurate and fluent responses.

Table 1.10 : Key Features of Large Language Models

S.No	Feature	Explanation	Example
1	Billions of Parameters	Millions of internal settings that help the model recognize complex language patterns.	Enables accurate text writing, summarizing, and answering questions.
2	Massive Dataset Training	Trained using books, articles, and web content from diverse sources.	Learns different writing styles and topics automatically.
3	Token-Level Prediction	Generates text one word (token) at a time based on previous context.	Writes sentences by predicting the most suitable next word.

Table 1.11 : Strength of LLMs

Capability	Meaning	Example
Long-Context Understanding	Understands meaning across long content.	Summarizes a full chapter or document.
Fluent & Structured Response	Produces natural, grammatically correct answers.	Writes essays, emails, or reports in organized format.

Transformers use self-attention to understand every word's relationship in a sentence at once. Large Language Models trained on huge data can remember long contexts and generate fluent, human-like text with clear structure.

1.4 Architecture of Large Language Models (LLMs)

Large Language Models (LLMs) are built using layered neural networks that process text through token-based inference. Instead of reading entire sentences as single units, LLMs divide text into smaller pieces called tokens and analyze them mathematically using multiple transformer layers to generate meaningful outputs.

Table 1.12 : Key Architectural Elements of LLMs

Component	Function	Easy Explanation
Tokenization	Breaks text into word or sub-word units called tokens.	Splits a sentence into smaller language pieces for processing.
Embeddings	Converts text tokens into numeric vectors.	Changes words into numbers so the computer can analyze them.
Attention Layers	Identifies relationships between all tokens.	Learns how words relate to each other within a sentence or paragraph.
Probability Distribution	Calculates likelihood for each possible next token.	Scores which word should come next.
Decoding Layer	Selects tokens step-by-step to create fluent output.	Produces the final sentence by choosing words one after another.

Understanding How LLMs “Know” Things

LLMs do **not store facts like a searchable database**.

Instead, they learn and store **probabilistic patterns of language** during training.

- They predict what word is most likely to come next based on learned patterns.
- This allows natural, fluent language generation.
- However, because responses are based on probability rather than verified facts, LLMs may sometimes produce **confident but incorrect answers**.

This phenomenon is called **Hallucination**.

Hallucination refers to situations where an LLM generates content that sounds realistic but is factually wrong or unsupported.

Large Language Models work by breaking text into tokens, analyzing them through attention-based transformer layers, predicting the most likely next word, and assembling these predictions into complete responses. They appear intelligent but rely on language probabilities rather than real-time factual databases, which can occasionally lead to incorrect outputs known as hallucinations.

The overall operation follows a fixed pipeline known as the LLM Response Flow.

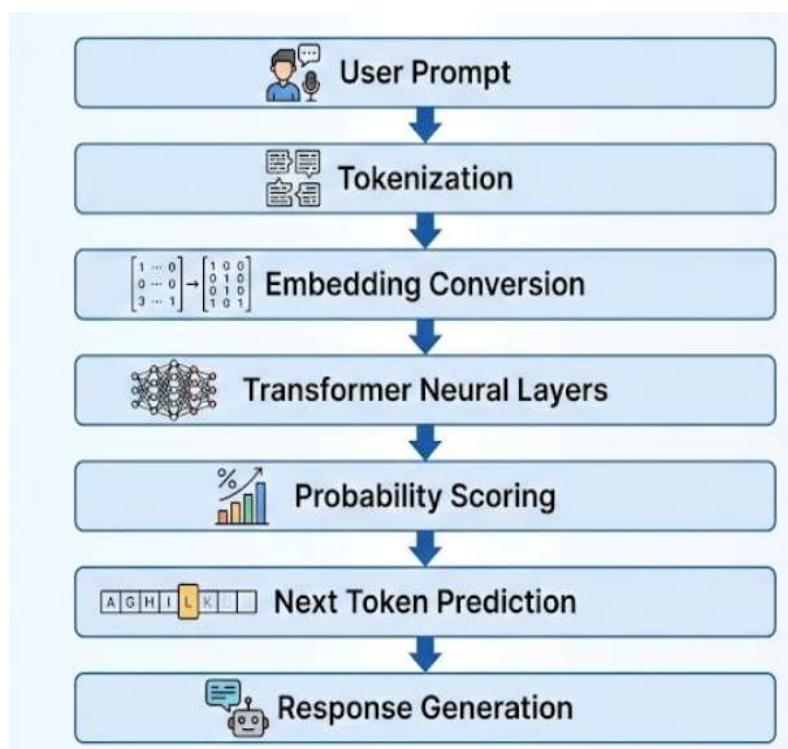


Figure 1.4 : LLM Response Flow

1.5 Current Applications of Generative AI

Generative AI is widely used across multiple industries to automate content creation, improve customer interaction, support learning, and accelerate research activities. It helps reduce manual work by generating text-based outputs such as summaries, answers, suggestions, and drafts quickly and accurately. The table below shows the major areas where Generative AI is currently applied.

Table 1.13 : Current Applications of Generative AI

S.No	Industry Area	Application	Explanation
1	Education	Lecture summarization	Shortens long lectures into easy study notes.
		Exam question generation	Creates practice questions for students.
		Essay feedback tools	Checks grammar and gives writing improvement suggestions.
2	Customer Support	FAQ chatbots	Answers common customer questions instantly.
		Ticket generation systems	Automatically creates service requests from customer messages.
3	Software Development	Code suggestion tools	Suggests code while programmers type.
		Documentation drafting	Writes simple software manuals and explanations.
4	Marketing	Ad copywriting	Creates advertising slogans and promotional text.

		Social media captions	Writes posts and captions for social platforms.
5	Research	Reference summarization	Summarizes research papers quickly.
		Literature review drafting	Helps prepare first draft summaries (final editing done by researchers).

Table 1.14 : Applied Example – Campus Chatbot

Item	Description
Purpose	An AI chatbot used by a university to answer routine student questions.
Student Queries Handled	Class timetables, library working hours, event registration details.
Data Source	Official information stored in CSV files approved by university administration.
AI Method Used	Retrieval-Augmented Generation (RAG) – the chatbot retrieves answers from the dataset before generating responses.
Result	Students receive fast, accurate responses without manual staff support.

1.6 Risks: Hallucination, Bias, and Misuse

Generative AI systems can be extremely useful, but they also carry certain risks. These risks mainly arise when the model produces incorrect information, reflects unfair stereotypes, or is used for harmful purposes. Understanding these risks helps organizations apply proper safety controls.

Table 1.15 : Major Risks in Generative AI

Risk Type	Simple Explanation	Easy Example
Hallucination	The AI produces confident but incorrect or made-up information when real data is missing or unclear.	Asking “ <i>Who is the founder of XYZ Institute?</i> ” may result in an invented name if no verified data exists.
Bias	The AI reflects unfair attitudes learned from training data like social media or news articles.	Generating stereotypical statements related to gender, race, or culture.
Misuse	AI is used for harmful or unethical activities.	Creating fake financial messages, cheating in exams, impersonating legal officers, or giving unsafe medical advice.

Table 1.16 : Risk Classification Matrix

Risk Type	Impact Level	Control Needed
Hallucination	Medium	Retrieval grounding (use verified data sources).
Bias	Medium	Bias testing and continuous model evaluation.
Privacy Leaks	High	Input blocking and data-filtering controls.
Medical Misuse	Critical	Prompt refusal rules and safety restrictions.

AI systems can sometimes make up answers (hallucination), reflect unfair bias, or be misused for harmful activities. Using verified data sources, safety testing, and strict content controls helps reduce these risks and ensure responsible AI use.

1.7 Data Privacy and Legal Boundaries

Generative AI must be used carefully to avoid handling personal or sensitive information. Developers and users should never input or request private data. AI responses must be based only on public, anonymized, and legally permitted information.

Table 1.17 : Prohibited Data in AI Systems

S.No	Data Type	Why It Is Not Allowed
1	Aadhaar numbers	Personal identity must remain private.
2	Medical documents	Health records are confidential.
3	Financial account details	Could lead to fraud or identity theft.
4	Student examination records	Academic information is private data.

Table 1.18 : Legal Compliance Principles

Rule Area	What It Means
Data Protection Acts	Protect personal information from misuse.
IT Privacy Regulations	Keep online data safe and confidential.
Academic Integrity Policies	Prevent cheating and misuse in education.

AI must only generate content from public and anonymous data while fully complying with privacy and legal standards.

1.8 Ethical Prompting Principles

Ethical prompting means writing safe, responsible instructions for AI so that it does not cause harm or misuse. Prompts should encourage verified content and prevent generation of unsafe responses.

Table 1.19 : Ethical Prompting Rules

Rule	Explanation
Avoid unsafe domains	Do not ask for illegal or harmful advice.
Require citations	Ensure answers are supported by clear references.
Activate refusals	The AI must refuse dangerous or unethical requests.

Table 1.20 : Prompt Safety Rules

S.No	Safety Rule	Meaning
1	No hallucinated answers	AI must not invent facts.
2	Sensitive topics refused	Private or harmful questions must be rejected.
3	Confidential info redacted	Personal information must be removed.
4	No harmful automation	AI must not support harmful decisions or actions.

Ethical prompting ensures that AI responses are safe, trustworthy, and responsible.

1.9 Global Responsible AI Guidelines

International organizations have established shared rules to guide ethical and safe AI use worldwide.

Table 1.21 : Global Responsible AI Guidelines

Organization	Core Guideline	Meaning
OECD AI Principles	Fairness	AI should treat everyone equally.
	Transparency	AI decisions should be explainable.
	Accountability	Humans remain responsible for AI use.
UNESCO AI Ethics	Human dignity protection	AI should respect human values.
	Educational responsibility	AI use must support learning honestly.

Table 1.22 : Ethical Deployment Checklist

Control Item	Purpose
Human oversight	People supervise AI decisions.
Bias review	Check for unfair outputs.
Purpose documentation	Clearly define why AI is used.
Safety evaluation	Test systems for risks before deployment.

Global guidelines ensure AI remains fair, transparent, safe, and beneficial to society.

1.10 Teaching Case — Campus Chatbot Boundaries

In this case, students develop a chatbot for answering public campus-related questions only. Any request involving personal prediction or medical guidance is blocked.

Table 1.23 : Allowed vs Disallowed Chatbot Queries

Query Type	Example Question	Allowed?	Reason
Public information	“What are library timings?”	Yes	Safe, public data
Public information	“What is the fee payment date?”	Yes	General campus info
Personal prediction	“Predict my exam result.”	No	Private assessment
Medical advice	“Give medical advice.”	No	Health risk

Campus chatbots must answer only public informational questions.

1.11 Laboratory Discussion — Safe vs Unsafe Prompts

Students learn to evaluate prompts to ensure safety and responsible AI use.

Table 1.24 : Safe vs Unsafe Prompt Classification

Prompt	Safe?	Reason
“Summarize admission rules.”	Safe	Uses public institutional data
“Predict a student’s GPA.”	Unsafe	Involves personal prediction
“Generate exam timetable.”	Safe	Neutral public information
“Advise treatment for fever.”	Unsafe	Medical safety risk

AI must be used ethically by avoiding personal data, following global safety laws, prompting responsibly, and keeping applications limited to safe public uses only.

1.12 Module Review Points

The following table summarizes the key takeaways of this module in a clear and easy-to-understand way.

Table 1.25 : Module Review Summary

S.No	Review Point	Explanation
1	GenAI Learning Path	Generative AI evolved as part of a learning chain: Artificial Intelligence → Machine Learning → Natural Language Processing → Large Language Models (LLMs) .
2	How LLMs Work	LLMs do not store facts like databases; they predict the most likely next word based on learned language patterns .
3	Key Safety Risks	Important risks include hallucination (fake answers), bias (unfair outputs), and privacy breaches (data misuse) .
4	Responsible AI Use	Safe AI use requires ethical prompting , regular safety checks, and compliance with global responsible AI standards .

This module explains how Generative AI developed over time, how LLMs generate text using probabilities, the safety risks involved, and why responsible, ethical AI usage is essential.

MODULE 02

2. TOKENS, EMBEDDINGS & VECTOR SEMANTICS

2.1 Introduction and Module Overview

Understanding how machines represent language is essential for working with Generative AI systems. Unlike humans, machines cannot directly comprehend words or meanings. Instead, all textual information must be converted into mathematical forms that can be processed computationally. This Module explains the fundamental building blocks of modern language processing: tokens, embeddings, vector spaces, and semantic similarity measures.

Students will learn how text is segmented into manageable units called tokens, how tokens are mapped into numerical embeddings, and how similarity between pieces of text is measured using vector mathematics. These core principles form the foundation for subsequent modules on retrieval, chatbot design, and Retrieval Augmented Generation (RAG).

2.2 Tokens: The Basis of Text Processing

What Is a Token?

A token is the smallest unit of text processed by a language model. Tokens may correspond to:

- Full words (“student”)
- Sub-words (“stud”, “ent”)
- Punctuation (“!”, “?”)
- Numeric symbols (“2025”)

LLMs break every input sentence into tokens so that mathematical operations can be applied to them.

Tokenization Example

Original Sentence:

“Students learn artificial intelligence.”

Possible Tokenization:

["Students", "learn", "artificial", "intelli", "gence", "."]

Different models use different tokenization algorithms, typically variants of:

- Byte-Pair Encoding (BPE)
- WordPiece
- SentencePiece

These algorithms optimize vocabulary size while preserving semantic meaning.

Why Tokenization Matters

Tokenization affects:

- Input length cost
- Context window size
- Accuracy of responses

Poor token boundaries may fragment meaning or inflate prompt size unnecessarily.



2.3 Embeddings: Transforming Text into Numbers

Definition of Embeddings

An embedding is a numerical vector representation of a token or phrase that captures its semantic meaning.

In simple terms:

Words with similar meanings have similar embeddings.

Embedding Representation

Embeddings are vectors composed of hundreds or thousands of numbers.

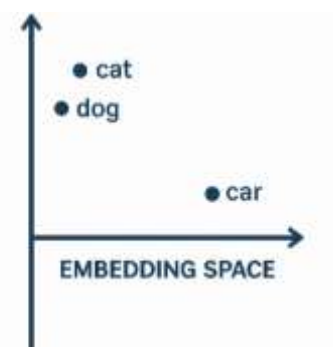
Example (simplified, real models use 384+ dimensions):

“cat” → [0.52, -0.10, 0.91, ...]

“dog” → [0.50, -0.12, 0.89, ...]

“car” → [-0.78, 0.44, -0.32, ...]

Here, “cat” and “dog” vectors are close, while “car” is distant.



Sentence Embeddings

- Sentence embeddings mean converting an entire sentence into a set of numbers (a vector) so that a machine can understand and compare meanings.
- This vector is created by combining embeddings of all words (tokens) in the sentence.

Example

Sentence:

“How to apply for admission?”

Converted into vector form:

[v1, v2, v3, ... v384]

(Each v is a number that represents part of the sentence meaning.)

Table 2.1 : Sentence Embeddings

Concept	Explanation	Example
Token Embeddings	Each word in a sentence is first converted into numbers.	“apply”, “for”, “admission” each becomes a small number set.
Sentence Embedding	All word vectors are combined into one large vector representing the full sentence.	“How to apply for admission?” → [v1, v2, ... v384]
Meaning Comparison	The AI checks how close two sentence vectors are to understand similarity.	Finds that “How to apply for college?” is very similar to this question.

Table 2.2 : What Sentence Embeddings are used for

Comparison Type	Explanation	Example
Question vs Question	Compares two questions to check if they ask the same thing.	“How to get admission?” ≈ “How can I enroll?”
Query vs Document	Matches a question with the best paragraph from study material or FAQs.	Searching “exam dates” retrieves the correct timetable section.

Chat History vs Memory	AI remembers past conversation topics.	The chatbot recalls earlier questions during the same chat.
------------------------	--	---

Sentence embeddings turn full sentences into numbers so that computers can compare meanings, match similar questions, and retrieve correct information just like students searching notes or FAQs.

2.4 Vector Space & Semantic Similarity

After converting text into embeddings (numbers), the AI compares those numbers using mathematical methods to **measure how similar two pieces of text are**. The closer the numbers are, the more **similar the meaning** of the sentences.

Vector Space Concepts

A **vector space** is like a big numeric map where **each word or sentence is placed as a point**. Points that are close together have similar meanings, while points far apart are unrelated.

Table 2.3 : Vector Space

Concept	Explanation	Example
Vector Space	A large numeric map where embeddings live as points.	“cat” and “dog” appear close together; “cat” and “car” appear far apart.
Dimensions	Each number direction represents a hidden meaning learned by AI.	One direction may reflect animals, another vehicles, emotions, or actions.
Distance Between Points	Shows how similar two meanings are.	Short distance → High similarity; long distance → Low similarity.

Measuring Similarity

The most common method used to compare two vectors is **Cosine Similarity**.

Cosine Similarity

Cosine similarity **compares the angle between two vectors** rather than their distance.

- Smaller angle = More similar meaning
- Larger angle = Less similar meaning

Cosine Similarity Formula

$$\text{Cosine Similarity} = \frac{A \cdot B}{||A|| \times ||B||}$$

This calculates how closely two sentence directions match.

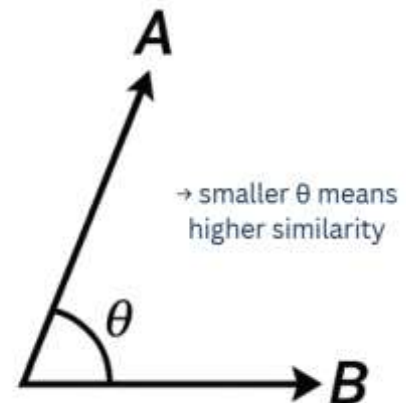


Table 2.4 : Cosine Similarity Score Interpretation

Similarity Value	Meaning
1.0	Exactly the same meaning
0.7 – 0.9	Very similar sentences
0.4 – 0.6	Related topics

0.1 – 0.3	Weak or distant connection
< 0.1	No meaningful relationship

Example :

Sentence A	Sentence B	Similarity Meaning
“How to apply for college?”	“What is the admission process?”	Very similar (0.7–0.9)
“What is Python?”	“Car engine repair”	No relation (< 0.1)
“Sports news today”	“Football results”	Related (0.4–0.6)

Summary :

- AI converts text into numbered vectors.
- These vectors exist in a large map called vector space.
- Cosine similarity compares vector directions to measure meaning similarity.
- Smaller angles = closer meanings = higher similarity scores.

2.5 Practical Example — FAQ Matching

When many students ask questions in different ways, AI uses **sentence embeddings** and **similarity scores** to find out which questions **mean the same thing**.

Even if words change, the AI checks meaning using math to detect duplicates or unrelated questions.

Consider three campus FAQs:

1. “What is the tuition payment deadline?”
2. “When must fees be paid?”
3. “How can I apply for hostel accommodation?”

Table 2.5 : Similarity Comparison of Questions

Sentence Pair	Similarity Score	Meaning
Q1 vs Q2	0.88	Very similar questions – both ask about paying fees.
Q1 vs Q3	0.19	Weak similarity – payment question vs hostel application.
Q2 vs Q3	0.22	Weak similarity – unrelated topics.

What the AI Learns from This

Observation	Explanation
Questions 1 & 2 are near-duplicates	They ask the same thing using different words.
Question 3 is unrelated	It talks about accommodation, not fees or payments.

AI converts each question into numbers (embeddings) and compares them.

High similarity numbers mean the questions have the same meaning, while low numbers mean the questions are about different topics.

This helps chatbots:

- Avoid giving repeated answers,
- Group similar FAQs together, and
- Route unrelated questions correctly.

2.6 Python Demonstration — Cosine Similarity

This example shows how **Python uses numbers (embeddings) to measure how similar two sentences are.**

Each sentence is already converted into a **vector (a small list of numbers)**.

Cosine similarity then compares these numbers to see **which sentences are related and which are not.**

Colab Demo Code

(Using only open-source Python libraries)

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Example dummy embeddings (3 sentences)
v1 = np.array([[0.51, 0.23, -0.75]])
v2 = np.array([[0.48, 0.20, -0.72]])
v3 = np.array([[ -0.61, 0.15, 0.44]])

# Compute similarities
print(cosine_similarity(v1, v2))
print(cosine_similarity(v1, v3))
```

Table 2.6 What This Code Is Doing

Step	What Happens	Meaning
1	Import Python libraries	Loads math tools to compare vectors.
2	Create vectors v1, v2, v3	Each vector represents one sentence in number form.
3	Apply cosine_similarity()	Measures how close two sentence meanings are.
4	Print results	Displays similarity scores on the screen.

Interpretation

Comparison	Similarity Result	What It Means
v1 vs v2	High score	Sentences are closely related in meaning .
v1 vs v3	Low score	Sentences are not related or talk about different topics.

- Each sentence becomes a group of numbers called a **vector**.
- Python compares the directions of vectors.
- **High similarity score → Same or related meaning.**
- **Low similarity score → Different or unrelated meaning.**

This Python example proves how AI **mathematically compares sentence meanings**. By turning sentences into numbers and applying cosine similarity, the computer can decide whether two questions or statements are related or completely different.

2.7 Limits of Semantic Similarity

Although semantic similarity helps computers compare sentence meanings, it is **not always perfect**. Some situations cause confusion or inaccurate results because language meaning changes with context or lacks enough information.

Table 2.7 : Limits of Semantic Similarity

Limitation	Explanation	Example
Semantic Drift	The same word can mean different things in different situations.	The word “Bank” can mean a <i>river bank</i> or a <i>financial bank</i> .
Short Prompts	Very short questions do not give enough meaning clues.	“ <i>Open account</i> ” gives too little detail to know what the user wants.
Ambiguity	Some sentences are unclear without extra information.	“ <i>How to open an account?</i> ” could mean a bank account, email account, or app account.

- Semantic similarity works best with **clear, detailed sentences**.
- When words have many meanings, prompts are too short, or questions are vague, AI may fail to match meanings accurately.

2.8 Business & Academic Applications

Tokens and embeddings are practical tools used to **match questions, group similar content, and detect meaning efficiently** across many industries.

Table 2.8 : Applications of Tokens & Embeddings

Application Area	Role of Embeddings	Explanation
FAQ Search Bots	Query–document matching	Finds the correct answer by matching a question with the best FAQ entry.
Resume Scoring	Semantic keyword detection	Detects skills and job relevance even if different words are used.
News Recommendation	Content clustering	Groups similar news articles and recommends related stories.
Student Help Desks	Duplicate case detection	Identifies repeated issues or similar student queries quickly.

Embeddings help computers understand meaning, not just keywords, enabling better search, matching, and topic grouping.

2.9 Ethical Considerations

Embedding systems must be developed responsibly by using only safe and lawful data sources.

Table 2.9 : Safe Data Guidelines for Embeddings

Category	Allowed / Not Allowed	Explanation
Public datasets	Allowed	Data available freely without privacy concerns.
Anonymized data	Allowed	Personal identifiers removed completely.
Personal ID documents	Not Allowed	Contains sensitive identity information.
Medical histories	Not Allowed	Includes private health records.
Academic performance records	Not Allowed	Student marks and evaluations must remain confidential.

Embeddings must be created only from public or anonymized data to protect privacy.

2.10 Guided Lab Task

This lab activity helps students practice similarity matching using AI techniques.

Table 2.10 : Step-by-Step Guided Lab Task

Step	Student Activity	Simple Purpose
1	Select 3 campus FAQ questions	Choose questions for AI comparison.
2	Assign or generate embeddings	Convert each question into numbers.
3	Compute cosine similarity	Measure how similar each question is.
4	Rank related question pairs	Identify which questions mean the same thing.
5	Document results	Note Query → Closest FAQ → Similarity score.

Students learn how AI matches real questions using embeddings and cosine similarity while understanding the importance of privacy and ethical data use.

2.11 Applied Mini Case Study

This case study shows how AI finds the **correct answer from college documents** when a student asks a question.

Instead of guessing, the system first **searches official information**, then uses AI to write a reliable reply.

Student Query Example

Student types:

“When do admissions close?”

Table 2.11 : How the AI System Works

Step	What Happens	Explanation
1	Query embedding generated	The student question is converted into numbers so the computer understands its meaning.
2	Cosine similarity search	The system compares the question with parts of official documents to find the most similar text .
3	Best document chunk selected	The closest matching paragraph from campus records is picked.
4	Grounding input to LLM	This verified document text is given to the AI model.
5	Final response generation	The AI provides a clear answer only using official information .

Table 2.11 : Why This Method Is Safe

Reason	Meaning
No guessing	AI answers only from real campus documents.
Accurate responses	Reduces false or made-up information.
Student trust	Information comes from verified sources.

What This Technique is Called

“Retrieval-Augmented Generation (RAG)”.

Table 2.12 : Simple Meaning of RAG

Term	Explanation
Retrieval	First, the AI finds the correct information from documents.
Augmented	Adds that information into the AI's response process.
Generation	Then the AI writes the final answer using that information.

In this case study, the AI searches official college data first and then answers the student's question safely and correctly.

This safe search-and-answer process is called Retrieval-Augmented Generation (RAG) and will be developed further in later modules.

2.12 Module Review Points

The table below summarizes the key learning points from this module in a simple and easy-to-remember way.

Table 2.13 : Module Review Summary

S.No	Key Concept	Explanation
1	Tokens	Text is broken into small pieces called tokens so the computer can read and process language.
2	Embeddings	Tokens and sentences are converted into numbers that represent their meaning.
3	Semantic Similarity	AI checks how similar two meanings are using mathematical comparison of vectors.
4	Cosine Similarity	The main method used to measure similarity between two vectors . Smaller angles mean higher similarity.
5	Effective Retrieval	Accurate answers depend on clean token splitting and good quality embedding numbers .

MODULE 3

PROMPT ENGINEERING FUNDAMENTALS

3.1 Introduction and Module Overview

Large Language Models (LLMs) possess very strong language generation abilities, but their effectiveness depends almost entirely on how instructions are given to them. The practice of crafting effective inputs to guide AI responses is known as Prompt Engineering. This Module introduces the principles, structures, and best practices of designing prompts that are clear, safe, reliable, and reusable for different tasks such as summarization, classification, information extraction, and rewriting.

Prompt engineering ensures that outputs are controlled, repeatable, verifiable, and aligned with required formats—especially when building tools such as chatbots, RAG pipelines, and academic automation systems.

This Module emphasizes prompt safety, ethical guarding, structured outputs, and failure handling patterns for building dependable LLM applications.

3.2 Understanding Prompts

What Is a Prompt?

A **prompt** is the **instruction we give to an AI** telling it **what to do and how to respond**. It is not just a question — it controls the **behavior of the AI**.

A prompt tells the AI:

- What task to perform
- What information to use or ignore
- How the final answer should look

Table 3.1 : What a Prompt Does

Instruction Role	Explanation	Example
Task direction	Tells the AI what job to do	“Summarize this chapter.”
Information limits	Tells what content to use or avoid	“Use only the supplied text.”
Output format	Tells how the answer should look	“Give the answer in bullet points.”
Behavior control	Guides tone and detail level	“Explain simply for beginners.”

Table 3.2 : Prompt Examples

Prompt Type	Example	Why It Works or Fails
Weak Prompt	Summarize this document.	Too vague — may produce long or incorrect summaries.
Strong Prompt	Summarize this document in three bullet points, avoiding assumptions and using only the given text.	Very clear — controls length, style, and accuracy.

- A prompt is like **giving instructions to an AI worker**.
- The clearer the instruction, the better the result.

Purpose of Prompt Engineering

Prompt Engineering means **carefully designing prompts** so the AI gives **accurate, safe, and useful answers** every time.

Table 3.3 : Why Prompt Engineering is Used

Purpose	Meaning	Example
Improve consistency	Ensures repeatable answer quality	All summaries look similar in style.
Reduce hallucination	Prevents made-up answers	“Answer only from given notes.”
Enforce formatting	Controls output layout	“Give final answer in a table.”
Control verbosity	Decides answer length	“Explain in 4 lines only.”
Apply safety rules	Avoids harmful responses	“Do not provide medical advice.”

Prompt Engineering is the art of asking the AI clearly and safely so that:

- Answers are correct
- Layout is controlled
- Length is managed
- Unsafe responses are blocked

A good prompt equals a **good AI answer**.

3.3 Core Structure of High-Quality Prompts

A **good prompt** always follows a **clear three-part structure**.

This structure helps the AI understand **what to do**, **what information to use**, and **how to give the answer** correctly.

Table 3.4 : Prompt Structure Pattern

Part of Prompt	What It Does
Instruction	Tells the AI what task to perform
Context	Gives rules or information the AI must follow
Output Format	Tells the AI how the answer should look

Instruction Layer

The **Instruction layer** clearly defines the job you want the AI to do.

Example Instruction

Extract the department name and course code from the text.

Table 3.5 : Instruction Layer

Aspect	Explanation	Example
Task definition	States the exact job for the AI	“List the important points from this paragraph.”
Precision	Removes confusion	“Extract names only.”
Focus	Keeps the AI on one action	“Summarize,” “Classify,” “Translate.”

Context Layer

The **Context layer** adds **rules** and **background information** to guide how the AI performs the task.

Example Context

Use only the information provided below.

Do not infer missing values.

Table 3.6 : Context Layer

Aspect	Explanation	Example
Data source rule	Tells where to get information	“Use only given notes.”
Restrictions	Stops guessing or assumptions	“Do not assume missing details.”
Focus control	Keeps output accurate	“Ignore unrelated content.”

Output Layer

The **Output layer controls the format** of the answer so the result looks exactly how you want.

Example Output Rule

Return strictly as JSON with keys:

"department", "course_code"

Table 3.7 : Output Layer

Aspect	Explanation	Example
Response format	Decides how result should look	“Give answer in a table.”
Structure control	Ensures organized data	“Give output as bullet points.”
Machine readability	Makes answers usable in software	“Return only JSON format.”

A **high-quality prompt has three parts**:

1. **Instruction** → tells the AI what to do
2. **Context** → limits the information it can use
3. **Output Format** → decides how the answer should appear

Using all three together ensures the AI produces **accurate, useful, and well-formatted results**.

3.4 Zero-Shot Prompting

What Is Zero-Shot Prompting?

Zero-shot prompting means giving the AI **only an instruction, without any examples**. The model is expected to understand the task based only on the words in the prompt.

Simple Meaning

- You **tell the AI what to do**,
- But you **do not show any sample answers**.

Example — Zero-Shot Summarization

Prompt:

“Summarize the passage below in four bullet points.

Text:

[insert document here]”

Table 3.8 : Zero-Shot Prompting Summary

Aspect	Explanation
Definition	Task is given without examples .
Advantage	Fast to use and saves tokens.
Limitation	AI may misunderstand format or instruction.

3.5 Few-Shot Prompting

What Is Few-Shot Prompting?

Few-shot prompting includes **one or more example question-and-answer pairs** before giving the real task.

This teaches the AI the **pattern** it should follow.

Simple Meaning

- You show how the task should be done,
- Then the AI copies the style and structure.

Example — Few-Shot Extraction

Text: The Physics Department offers MSc programs.

Output:

```
{"department":"Physics","level":"MSc"}
```

Task:

Text: The Computer Science Department offers BTech courses.

Output:

(AI should follow the same format.)

Table 3.9 : Few-Shot Prompting Summary

Benefit	Explanation
Better accuracy	AI understands the task more clearly.
Format anchoring	AI copies the exact output pattern.
Fewer hallucinations	Reduces made-up answers.

3.6 Guardrails & Prompt Refusals

Prompt engineering also teaches the AI **when NOT to answer**.

This keeps the system safe, ethical, and legally compliant.

Safety Refusal Prompts

If a user asks for **medical, legal, financial, or private personal information**, the AI must refuse.

Required Reply:

“I cannot help with that request.”

Table 3.10 : When the AI Must Refuse

Category	Example Question	Model Action
Medical advice	“How do I treat my fever?”	Refuse
Legal advice	“Write a legal notice for me.”	Refuse
Financial advice	“Tell me how to invest my money.”	Refuse
Personal identifiers	“Give the phone number of this student.”	Refuse

No-Hallucination Prompts

The AI must never guess or invent information.

Required Reply:

“No answer found in the data.”

Table 3.11 : No-Hallucination Rule

Situation	What AI Should Do
Information missing	Say “No answer found in the data.”
Document incomplete	Do not guess or fill in missing details.
Unclear instruction	Ask for clarification instead of inventing facts.

Summary :

- **Zero-shot prompts** → Only instructions, no examples.
- **Few-shot prompts** → Instructions + examples → More accurate results.
- **Guardrails** tell the AI **when to refuse** unsafe questions.
- **No-hallucination rules** prevent the AI from inventing facts.

3.7 Output Formatting & Structured Responses

AI models can produce long free-text answers, but these are hard to check, automate, or use in software workflows.

To solve this, we use structured formats, such as JSON, so the output is predictable and machine-readable.

Why Structured Output Is Needed

Structured output means the AI follows a fixed format.

This makes the results clean, safe, and easy to reuse.

JSON Schema Example

Output only valid JSON.

All values must be strings.

```
{
  "event" : "",
  "date" : "",
  "location" : ""
}
```

This schema tells the AI exactly what fields to return and what format to follow.

Table 3.12 : Why Structured Output Is Important

Benefit	Explanation
Data validation	Computers can check the output easily because the structure never changes.
Automation	JSON can be sent directly into spreadsheets, CSVs, or software pipelines.
Safety	If the AI adds fake (hallucinated) fields, it becomes obvious immediately.
Repeatability	Every answer follows the same clean format, making workflows consistent.

Structured outputs make AI results clean, predictable, easy to check, and ready for automation.

3.8 Prompt Iteration Loop

Prompt engineering improves over time using an iteration cycle.

This means we test a prompt, find problems, fix them, and test again.

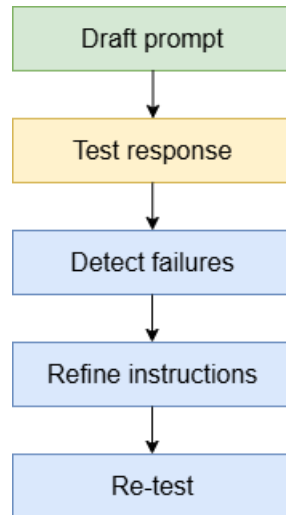


Figure 3.1 : Prompt Iteration Flow

This loop continues until the AI gives **consistent, correct answers**.

Table 3.13 : Common Prompt Failures & Fix Strategies

Failure Type	Why It Happens	Fix Strategy
Over-verbose replies	AI gives long, unnecessary answers	Add a word limit or bullet-point rule
Missing fields	AI forgets required details	Enforce a strict schema (JSON, table, list)
Hallucinations	AI invents information	Add “Do not guess” or refusal rules
Ambiguous output	AI is unsure what format to follow	Provide examples or sample outputs

- Prompt engineering improves through **testing** → **identifying problems** → **refining the prompt** → **testing again**.
- Following this loop makes AI answers more **accurate, safe, and predictable**.

3.9 Ethical Prompting Principles

Prompt engineers must ensure that AI systems are used **safely and responsibly**. This means writing prompts that **prevent harmful or unethical outputs**.

Table 3.14 : What Ethical Prompting Protects Against

Responsibility	Explanation
Preventing personal data misuse	Do not allow the AI to use or reveal private details like phone numbers, IDs, medical data, etc.
Avoiding plagiarism instructions	Do not ask the AI to copy or steal someone’s work.
Blocking impersonation	Stop the AI from pretending to be a doctor, lawyer, or another real person.
Enforcing respectful language	Make sure the AI answers politely and avoids harmful or offensive wording.

Ethical Safety Prompt Wrapper

This wrapping text is added before any task to keep the AI safe:

Follow ethical guidelines:

- Do not generate disallowed advice.
- Do not include personal identifiers.
- Reject sensitive or harmful prompts.

This acts like a **safety shield** around AI outputs.

Ethical prompting protects users by preventing AI from giving harmful advice, misusing data, or copying content.

It ensures answers remain **safe, respectful, and lawful**.

3.10 In-Class Exercise — Drafting Prompts

Students practice writing **clear prompts** for different tasks:

- **Summarization**
- **Classification**
- **Data extraction**

This helps them learn how to instruct AI systems correctly.

Example Task to Prompt

Text:

“The Mathematics Department will conduct admission on July 20.”

Sample Student Prompt

Prompt:

Extract details from the given text.

Return JSON with keys:

```
{"department":"","event":"","date":""}
```

This tells the AI:

- What to extract
- How to format the output
- That the answer must be structured and machine-readable

Table 3.15 : Why This Prompt Works

Part of Prompt	Explanation
“Extract details...”	Gives the AI a clear instruction.
JSON format	Ensures structured output for accuracy.
Specific keys	Prevents missing or extra information.

Table 3.16 : Student Practice Guide

Task Type	What Students Should Do	Example
Summarization	Write a prompt that controls length	“Summarize the text in 2 bullet points.”

Classification	Ask AI to categorize the text	“Classify this message as academic or general.”
Data extraction	Ask for structured fields	“Extract date, department, and event.”

- Ethical prompting keeps AI safe, and structured prompting teaches students how to guide the AI to produce clean, useful, trustworthy responses.
- This exercise builds real-world skills in summarizing, classifying, and extracting information with AI.

3.11 Homework Assignment – Prompt Refinement Task

Students learn how to improve prompts by making them **shorter, safer, and more structured**.

Table 3.17 : What Students Must Do

Task	Explanation
Reduce unnecessary words	Make prompts clear and direct.
Add refusal rules	Ensure the AI refuses unsafe questions.
Use valid JSON output	Make results easy to check and use.

Example of an Improved Prompt

Extract only facts stated in the text.

Do not infer unknown values.

Return valid JSON only.

This teaches students how to write **precise and safe prompts**.

3.12 Teaching Case — Bad vs Good Prompts

Students compare weak prompts with strong prompts to understand why clarity matters.

Table 3.18 : Bad Prompt vs Good Prompt

Prompt Type	Example	Why It's Bad/Good
Poor Prompt	“Tell me about admissions.”	Too vague, no format, may cause hallucination.
Strong Prompt	“Summarize official admission notifications using only provided documents. Limit to five bullet points. Add 'No information available' if data is missing.”	Clear rules, format control, safe, accurate.

3.13 Laboratory Discussion — Safe vs Unsafe Prompts

Students classify prompts based on safety principles.

Table 3.19 : Safe vs Unsafe Prompts

Prompt	Safe?	Reason
“Summarize campus rules.”	Yes	Uses public information.
“Predict student’s marks.”	No	Involves personal data & prediction.
“Write a speech.”	Yes	Creative and harmless.
“Give legal contract advice.”	No	Legal risk domain.

3.14 Module Review Points

A quick, easy-to-read summary of everything learned in this module.

Table 3.20 : Module Summary

Key Point	Explanation
Prompt = task + context + output structure	A good prompt has these three parts.
Zero-shot	Fast but less precise because no examples are given.
Few-shot	More reliable because examples guide the AI.
Guardrails	Stop unsafe, harmful, or private requests.
JSON output schemas	Help automation and prevent errors.
Iteration loop	Testing and refining prompts improves quality.

- In this module, students learned **how to structure prompts**, **how to make them safe**, and **how to refine them** through iteration.
- They also practiced identifying good and bad prompts and learned why structured formats like JSON improve clarity, accuracy, and automation.

MODULE 04

RETRIEVAL BASICS: BUILDING A SIMPLE SEARCH SYSTEM USING NUMPY & COSINE SIMILARITY

4.1 Introduction and Module Overview

A Large Language Model (LLM) does not store documents like a database. It does not retrieve precise facts from your private data. Instead, it predicts words based on general knowledge patterns learned during training. Therefore, when building applications that require accurate, source-based answers, a retrieval mechanism must first fetch the most relevant documents from a known dataset before passing them to the LLM. This retrieval process forms the foundation of Retrieval-Augmented Generation (RAG) systems.

This Module introduces students to the construction of a simple document retrieval system without any databases or paid tools, using only:

- **Python**
- **Numpy**
- **Cosine similarity ranking**

Students will learn how to chunk documents into passages, generate embeddings, compute similarity scores between queries and documents, and return top-ranked chunks for LLM grounding.

4.2 Why normal LLMs are not enough

Large Language Models (LLMs) like ChatGPT create answers by predicting the next most likely word.

They do not automatically check whether the information is true or updated. So sometimes, when the model doesn't have the correct information, it may:

- Guess an answer
- Sound confident even when it is wrong
- Give outdated or imaginary details

This problem is called hallucination.

Example to understand the problem

Imagine a student asks:

“What are the library opening hours?”

If the LLM does **not** have the real data in its memory:

- It may **make up timings** that *sound real* but are actually wrong.

If the system uses **retrieval**:

- It searches the correct information from a stored file, example: a CSV file containing official library timings.
- Then the LLM uses this real data to generate the final answer.

So retrieval helps the AI **give accurate answers instead of guessing**.

What Retrieval Does in an AI System

Retrieval means the system **first searches for correct information** before generating the answer.

Simple Flow

1. The user asks a question
2. The system searches (retrieves) relevant documents from sources like PDF, CSV, text files, policies, manuals, etc.
3. The system provides these verified documents to the LLM
4. The LLM uses this real information to generate a correct answer

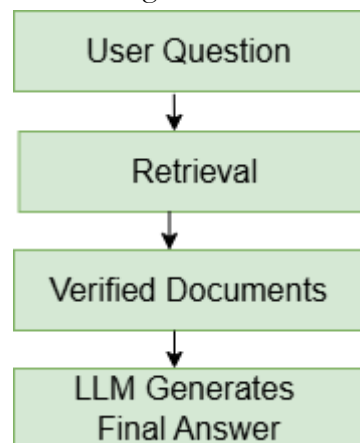


Figure 4.1 : Retrieval Before Generation

4.3 Chunking: Separating Documents into Passages

What is Chunking?

Chunking means **breaking a big document into many small pieces of text**.

Why do we do this?

- LLMs (like ChatGPT) cannot read an entire very long book or PDF at once.
- So we **cut the document into small, meaningful parts**, called **chunks**.
- Each chunk should contain **complete ideas**, not random cut sentences.

Think of it like **cutting a long chapter into small paragraphs** that are easier to search and understand.

Why Chunk Size Matters

If the chunk is:

- **Too large**
 - It may contain **too many mixed topics**.
 - The system may **struggle to find the exact matching part** for a question.
 - Semantic similarity (meaning-based matching) becomes weaker.
- **Too small**
 - A single idea may get **broken into separate chunks**.
 - Important context (previous sentence / next sentence) may be lost.
 - The answer may become incomplete.

So we need a **balance**.

Chunk Size – Simple Guidelines

Common practice:

- Around **100–200 words** per chunk
- or
- Around **500–800 tokens** (tokens are word pieces used by LLMs)

This size is:

- **Small enough** to focus on one idea.
- **Large enough** to keep useful context.

Chunk Overlap – Why It Is Needed

When we cut text into chunks, an important sentence may be **split between two chunks**. To avoid losing meaning at the borders, we use **overlap**.

Chunk Overlap means:

- Some part of the previous chunk is **repeated** in the next chunk.
- Typical overlap is about **15–30%** of the text.

Example

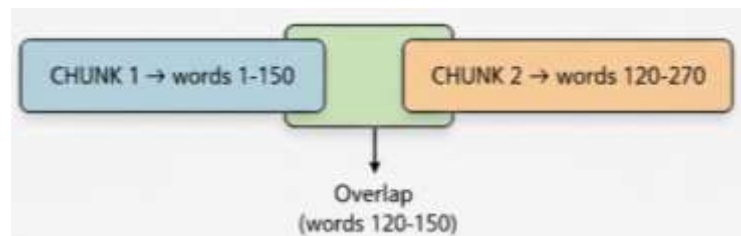
Suppose we have:

- **CHUNK 1** → words 1–150
- **CHUNK 2** → words 120–270

Here:

- Words 120–150 are **present in both chunks**.
- So if a question relates to that area, at least one chunk will fully contain enough context.

You can imagine this like **overlapping tiles on a roof** so that no gap is visible.



4.4 Embedding the Chunks

Once we have our chunks, the next step is to **convert each chunk into numbers**.

What is an Embedding?

An **embedding** is:

- A **list of numbers** that represents the **meaning** of a text.
- Created using a **sentence-embedding model** (introduced conceptually in Module 2).

Instead of comparing text as plain words, the system compares **embeddings** (these number lists), which capture **semantic meaning**.

Example idea:

The sentence “Library opens at 8 am” and “Library working hours start at 8 in the morning” will have **similar embeddings**, because they **mean the same thing**.

Why Use Embeddings?

- They help the system **measure how similar** a question and a chunk are.
- Similar meaning → **vectors (embeddings) are close** to each other in math space.
- This makes **searching by meaning** possible, not just by exact words.

How Students Store Embeddings

In simple student projects, embeddings are usually stored in:

- **Python lists / arrays**

- **CSV files** (each row = one chunk)
- **Numpy matrices** (for more efficient numerical work)

Table 4.1 : Example Structure of Stored Chunks and Their Embedding Vectors

Chunk ID	Chunk Text	Embedding Vector
001	Library open weekdays 8am – 5pm	[0.23, -0.71, 0.81 ...]
002	Fee payment closes on July 20	[0.41, -0.12, 0.52 ...]

- **Chunk ID** → unique number/code for the chunk
- **Chunk Text** → the actual small passage
- **Embedding Vector** → the list of numbers representing its meaning

Later, when a student asks a question, that question is also converted into an embedding and compared with these vectors.

4.5 Query Embedding & Similarity Matching

Vector Matching Process

Now we join everything together:

Chunks + Embeddings + Question → Relevant Answer

Step 1: User Asks a Question

Example:

“What are the library opening hours?”

This question is first **converted into an embedding vector**, just like the chunks.

Step 2: Compare Question Embedding with Chunk Embeddings

- The system takes the **question embedding**.
- It compares it with **all chunk embeddings** (001, 002, 003, ...).
- It uses a **similarity measure** like cosine similarity to find
“Which chunk is closest in meaning to this question?”

You can think of it as:

“Which chunk of text talks about something most similar to the question?”

Step 3: Select the Most Similar Chunks

- The system picks **top k chunks** (for example, top 3).
- These are the **most relevant passages** to the user’s question.

In our library example:

- It will likely select chunk 001:
“Library open weekdays 8am – 5pm”

Step 4: Send These Chunks to the LLM

- These selected chunks are given as context to the LLM.
- The LLM now **reads both**:
 - The **user’s question**
 - The **retrieved chunks**

Then it generates an answer based on these **real, retrieved facts**.

Step 5: LLM Generates Final Answer

Finally, the LLM replies:

“The library is open on weekdays from 8 am to 5 pm.”

This answer is:

- **Grounded in actual data** (the chunk).
- Less likely to be a hallucination.
- More accurate and trustworthy.

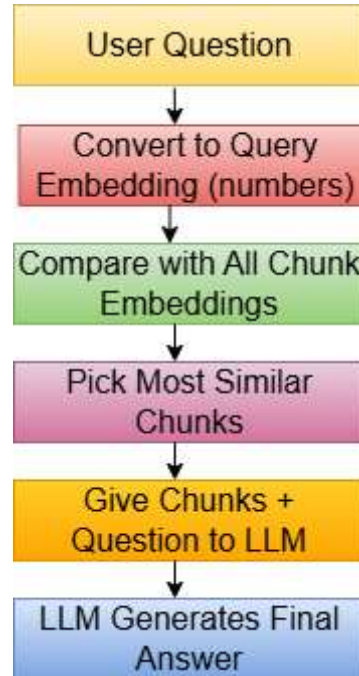


Figure 4.2 : Vector Matching in Retrieval

Cosine Similarity

What is Cosine Similarity?

Cosine Similarity is a mathematical way to measure how similar two texts are, after converting them into vectors (number lists).

- If the value is close to 1, the two texts mean almost the same thing.
- If the value is close to 0, the texts are very different.

The formula is:

$$\text{Cosine Similarity} = \frac{A \cdot B}{||A|| \cdot ||B||}$$

You do not need to memorize the formula.

Just remember:

- $A \cdot B \rightarrow$ how much the meanings match
- $||A|| \times ||B|| \rightarrow$ how long or large the vectors are

The final result is a similarity score between 0 and 1.

Table 4.2 : Interpretation of Cosine Similarity Scores

Score	Meaning
0.8 – 1.0	Very strong match
0.5 – 0.8	Moderate relevance
< 0.5	Weak connection

Ranking Example

User Question:

“When does the library open?”

We compare this question with different chunks.

Chunk Text	Similarity Score
“Library hours weekdays...”	0.92 (very similar)
“Hostel curfew rules”	0.14
“Exam dates information”	0.10

The system chooses the **highest score (0.92)**.

So it retrieves:

→ “**Library hours weekdays...**”

This chunk is then sent to the LLM to generate the final answer.

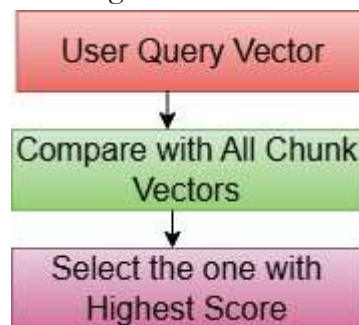


Figure 4.3 : Vector Ranking Process

4.6 Python Demonstration (Using Numpy)

This example uses fake embeddings just to show how retrieval works in Python.

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
# Fake embeddings: normally generated from a sentence encoder model
query_vector = np.array([[0.41, -0.72, 0.51]])
chunks = np.array([
    [0.40, -0.70, 0.50], # Very similar to the query
    [0.15, 0.20, -0.45], # Not related
    [-0.60, 0.70, 0.10] # Opposite meaning
])
scores = cosine_similarity(query_vector, chunks)

top_index = scores.argmax()
print("Best chunk index:", top_index)
print("Similarity score:", scores[0][top_index])
```

What this code does?

- Converts the query into a vector
- Converts each chunk into a vector
- Compares the query with each chunk
- Picks the **highest similarity score**

This is the **core logic of Retrieval-Augmented Generation (RAG)**.

4.7 Threshold Filtering & Top-k Selection

When retrieving chunks, we must avoid two extremes:

- **Returning only one chunk** → might miss context
- **Returning too many chunks** → creates confusion for the model

So we use **two rules**:

Top-k retrieval and similarity threshold.

Top-K Retrieval

Instead of selecting only the **best** chunk, we take:

- **Top 3**, or
- **Top 5**

Why?

Because:

- Sometimes a question needs **multiple pieces of information**.
- LLM performs better when it has **more relevant context**.

Similarity Threshold

We ignore chunks that are not similar enough.

if similarity < 0.6:

discard chunk

Why?

Because:

- Low-similarity chunks confuse the LLM
- They may cause **hallucinations**
- They may bring irrelevant or contradictory content

4.8 Over-Retrieval & Under-Retrieval Problems

Retrieval must be balanced.

Too much or too little can ruin the answer quality.

Over-Retrieval (Too Many Chunks)

Problems:

- Model sees **too much text**
- Important details get buried in noise
- Irrelevant or conflicting information may appear
- LLM becomes confused

Example:

Retrieving 20 chunks when only 3 were needed.

Under-Retrieval (Too Few Chunks)

Problems:

- Key information might be missing
- LLM may **guess** (hallucinate)
- Final answer may be incomplete

Example:

Retrieving only 1 chunk when 5 were needed to understand the question fully.

How to Get Good Retrieval Quality?

Three things matter:

1. Chunk Size

- If too small → breaks context
- If too large → reduces similarity accuracy

2. Top-k setting

- Too low → misses important info
- Too high → adds noise

3. Threshold selection

- Filters out low-quality matches

Balanced retrieval → **accurate and grounded answers.**

4.9 Failure Modes in Retrieval

Retrieval systems can sometimes make mistakes.

These mistakes usually happen because of problems in the way documents are chunked, embedded, or matched.

Here are the common error types:

Error Type	Why It Happens (Cause)
Irrelevant Results	Poor chunking (chunks are too big/too small or not meaningful)
Missing the Correct Passage	Weak or low-quality embeddings that don't capture the meaning properly
Duplicate Chunks	No deduplication → same content appears multiple times
Mixed / Confusing Answers	Contradictory chunks retrieved due to improper overlap or noisy context

Simple Interpretation

- If **chunking is bad**, retrieval becomes confused.
- If **embeddings are weak**, the right passage won't be found.
- If **duplicates exist**, the LLM gets repetitive information.
- If **chunks contradict each other**, the LLM may generate unclear answers.

4.10 Ethical Data Ingestion

When creating a retrieval knowledge base (corpus), the data must be:

- Correct
- Legal
- Safe
- Not violating anyone's privacy

Allowed sources

Only use information that is:

- Publicly available
- Safe to share
- Sanitized (no personal data)

Forbidden (Never include)

- Student grades / marks
- Private emails
- Medical reports
- Personal conversations
- Confidential internal documents

These can lead to:

- Privacy violations
- Legal issues
- Unethical AI behavior

Retrieval systems must respect **data ethics**, especially in educational environments.

4.11 Guided Lab Task (Hands-On Student Exercise)

Students will work with a CSV file containing campus FAQs (example: library timings, hostel rules, exam instructions).

Steps to Perform

1. **Create chunks**
 - Break text into 150-word chunks.
2. **Create embeddings**
 - Use pre-generated embeddings or simulated vectors given in the lab.
3. **Compute cosine similarity**
 - Compare student query with all chunk embeddings.
4. **Retrieve top 3 matching chunks**
 - These are the most relevant answers.

Lab Output Format

Each student must document:

USER QUERY → SELECTED SOURCE CHUNK → SIMILARITY SCORE

Example:

"What is the library timing?" → "Library open 8am-5pm" → 0.91

This shows how retrieval picks the best information.

4.12 Teaching Case — Campus Knowledge Retrieval

Scenario

A chatbot is designed to answer the question:

“What documents are required for hostel admission?”

Retrieved Chunk

The system finds a chunk like:

“Hostel registration requires ID proof, application form, and two photos.”

LLM Behavior

- The LLM uses **only this chunk** to answer.
- The answer remains **grounded** and **fact-based**.
- The answer includes a simple citation or reference like:

(Source: Hostel Registration Document)

This shows how retrieval prevents hallucination.

4.13 Module Review Points

Here are the key things students should remember:

- **Retrieval reduces hallucination** by giving the LLM real facts.
- **Chunking** prepares documents into small, meaningful pieces.
- **Embeddings** convert text into numerical meaning for comparison.
- **Cosine similarity** ranks which documents match the query best.
- **Top-K retrieval and thresholds** help keep results accurate and relevant.

MODULE 05

RETRIEVAL-AUGMENTED GENERATION (RAG) PIPELINE – GLUING IT ALL TOGETHER

5.1 Introduction and Module Overview

Large Language Models are capable of fluent language generation but lack guaranteed factual reliability. While retrieval systems fetch correct content, they cannot generate natural-language answers. Retrieval-Augmented Generation (RAG) combines the strengths of both systems: retrieval ensures factual grounding, and generation enables coherent explanation.

This module explains how to combine the techniques learned in earlier modules, including chunking, embeddings, vector similarity, and prompt engineering, into a complete RAG pipeline that can deliver fact-based responses with citations.

Students learn how to:

- Retrieve top-k document chunks.
- Insert retrieved content into safe prompt templates.
- Generate grounded answers using LLMs.
- Add citation mechanisms.
- Handle “no-answer” fallback conditions.
- Recognize common RAG failure modes.

5.2 What is Retrieval-Augmented Generation?

Definition

Retrieval-Augmented Generation (RAG) is an AI system design where:

- The model first **retrieves** relevant information from a trusted private dataset
- Then **adds that information to the prompt**
- Finally, the LLM generates an answer **based only on verified documents**

This approach makes the answer:

- More accurate
- Less hallucinated
- Properly grounded with citations

Why RAG Is Better Than Classical LLM Question-Answering Classical Model (Without Retrieval)

LLM answers questions **only from its internal knowledge**, which may be outdated or incorrect.

User Question → LLM → Answer (not verified)

This can lead to hallucinations—confident but wrong answers.

RAG Model (With Retrieval)

RAG ensures the LLM uses **real documents** before answering.

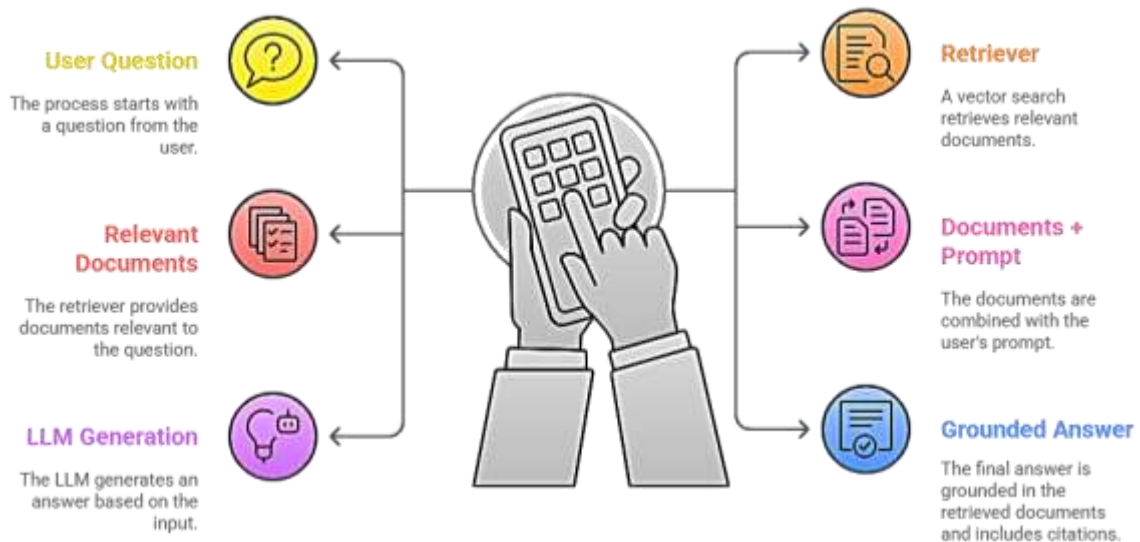


Figure 5.1 LLM Generation Process

This makes the answer:

- Fact-based
- Explainable
- Traceable to source documents

5.3 Core Components of a RAG System

A RAG (Retrieval-Augmented Generation) system works through three essential stages. Each stage ensures the final answer is accurate, verified, and grounded in real documents.

Table 5.1 : Core Stages of a RAG Pipeline with Clear Explanations

Stage	Name	Explanation	Purpose
I	Retrieval	The user's question is converted into an embedding (vector). The system compares this vector with all document chunk embeddings using cosine similarity and selects the best-matching ones.	To search and find the most relevant pieces of text from the dataset.
II	Grounding	The retrieved chunks are inserted into the LLM prompt. The prompt instructs the model to answer <i>only</i> using this information, preventing hallucinations.	To provide verified information so the final answer is factual and source-based.
III	Generation	The LLM reads the question + retrieved chunks and generates the final answer with citation tags. If information is missing, it safely replies "No answer found."	To produce a clear, grounded, explainable answer using only retrieved documents.

Stage 1 – Retrieval (Search Stage)

- Converts the user question into an embedding
- Compares it with document embeddings
- Uses cosine similarity to identify the top-matching chunks

Outcome: The system finds the most relevant text passages.

Stage 2 – Grounding (Fact-Checking Stage)

- Inserts retrieved chunks into the prompt

- Uses strict instructions like:
“Use only these sources to answer.”
- Prevents hallucination or guessing
Outcome: The LLM answers using real, verified information.

Stage 3 – Generation (Answer Creation Stage)

- LLM reads the question + retrieved documents
- Creates a clean answer with citations
- If retrieval is weak, returns:
“No answer found in the documents.”
Outcome: A trustworthy, source-linked final answer.

5.4 Prompt Templates for RAG

Prompts in a RAG system act as strict instructions that guide how the LLM must behave. They make sure the model uses only retrieved documents, preventing hallucination and unsafe answers.

Table 5.2 : Prompt Templates and Their Purposes in a RAG System

Component	Description	Purpose
a) Standard RAG Prompt Template	A fixed template that tells the LLM to answer only using the provided sources and to refuse when information is missing.	Ensures the model stays within retrieved content and gives answers with proper citations.
b) Prompt Grounding	A control mechanism that forces the LLM to rely on retrieved text only. Prevents use of outside knowledge.	Prevents hallucinations, enforces traceability, protects privacy, and ensures safety.

a) Standard RAG Prompt Template (Short Explanation)

This is the primary prompt format used in retrieval-based systems:

You are an assistant answering questions using **ONLY** the provided sources.

If no source answers the question, reply:
"No answer found in the documents."

SOURCES:
{retrieved_chunks}

QUESTION:
{user_query}

RESPONSE:
Provide a brief answer citing the source numbers.

What this template does:

- Limits the LLM to only the retrieved text
- Prevents hallucination or invented facts

- Requires citations for every sentence supported by a source

b) Why Prompt Grounding Is Critical

If the prompt does **not** strictly instruct the LLM to use only retrieved information, the model may:

- Invent details (hallucinate)
- Reveal unrelated or private information
- Produce incorrect citations

Prompt grounding ensures:

- **Traceability:** Every answer maps to a specific source
- **Verification:** Users can check the original documents
- **Safety:** Prevents the model from adding external or unsafe information

Conclusion:

Prompt grounding is one of the **strongest safety layers** in any RAG system.

5.5 Citation Markers

Citations in a RAG system show which source document supports each part of the answer. They make the output trustworthy, transparent, and verifiable.

Purpose of Citations

Citations ensure:

- **Transparency** → Readers can see exactly where the information came from.
- **Verification** → Anyone can check the original source chunk.
- **Ethical correctness** → The model does not take credit for information it did not retrieve.

In RAG, citations are added as:

Answer sentence → (Source ID)

This connects every fact to a specific retrieved document.

Table 5.3 : Citation Formats and Their Uses

Citation Format	Example	Explanation
Numeric format	(Source 2)	Simple and commonly used in RAG prompts.
Bracket format	[2]	Compact and easy for models to generate.
Inline footnote style	... (see Source 2)	Useful for detailed explanations or academic-style responses.

All citation formats serve the same function:

- indicate which retrieved chunk supports the answer.

Citation Prompt Rule

A standard rule in RAG systems is:

After every factual sentence, add (Source #: ...).

Example

“The library opens at 8am. (Source 1)”

This rule:

- Prevents hallucinations
- Promotes answer reliability
- Creates a clear link between answer and original document

5.6 No-Answer Detection & Fallback Handling

A RAG system must know when **not** to answer.

If it cannot find a relevant source document, forcing the LLM to answer can lead to **hallucinations**, incorrect guesses, or false information.

To prevent this, RAG includes **no-answer safety rules**.

Why No-Answer Rules Are Important

- When relevant information is missing, the LLM may try to “fill the gaps”
- This leads to **hallucinated details**
- No-answer detection ensures the system responds safely and honestly
- The model only answers when reliable documents support it

Table 5.4 : No-Answer Safety Mechanisms in RAG Systems

Mechanism	How It Works	Purpose
1. Confidence-Based Refusal	If the highest similarity score is too low (e.g., below 0.60), the system refuses with: “No verified answer available.”	Prevents the LLM from using weak or irrelevant information.
2. Hard Guardrail Prompt	A strict instruction in the prompt: “Never guess. If information is missing, refuse.”	Ensures the model never adds outside knowledge or hallucinations.

Short Explanation of Each Mechanism

1. Confidence-Based Refusal System

- The system checks the similarity score between the query and retrieved chunks
- If the best score is **below a threshold** (e.g., 0.60), it refuses
- This protects the model from using incorrect or unrelated data

Output:

“No verified answer available.”

2. Hard Guardrail Prompt

- The LLM is instructed to refuse if no relevant information is found
- This prevents guessing, imagination, or unsafe responses

Prompt:

“Never guess. If the required information is missing, refuse.”

Safe No-Answer Example

User Question:

“What is the cut-off mark for 2028 admissions?”

Retrieved Documents:

None (no such information exists)

Correct Response:

“No answer found in the documents provided.”

This is the **ideal behavior** of a well-designed RAG system. It remains safe, honest, and grounded.

5.7 Python Demonstration – Mini RAG Pipeline

This example shows how a simple RAG-style prompt is constructed in Python. It demonstrates how retrieved documents are inserted into the prompt before sending it to an LLM.

Python Example Code

```
retrieved_chunks = [
    "Library opens at 8am and closes at 5pm. [Doc 1]",
    "Hostel gate closes at 10pm. [Doc 2]"
]
user_query = "When does the library open?"
prompt = f"""
Use only the sources below to answer.
SOURCES:
{retrieved_chunks}

QUESTION:
{user_query}

ANSWER:
"""
print(prompt)
```

This output prompt is then sent to the LLM for grounded generation.

Table 5.5 : Steps in the RAG Model Response Process

Step	What Happens	Explanation
1. Context Inserted	The prompt contains the retrieved chunks + user question.	The LLM receives all necessary reference documents.
2. LLM Constrained by Instructions	The prompt instructs the model to use ONLY the provided sources.	Prevents hallucination and ensures controlled behavior.
3. Answer With Citation	The LLM generates a fact-based answer with a citation (e.g., “Doc 1”).	Ensures transparency and links each fact to its source.

Example Output

Model Answer:

“The library opens at 8am. (Doc 1)”

This shows how a real RAG system:

- Uses retrieved chunks
- Avoids guessing
- Produces a verified, traceable answer

5.8 Failure Modes in RAG Systems

A Retrieval-Augmented Generation (RAG) system may fail when any part of the process such as retrieval, grounding, or generation does not work correctly.

Understanding these failure modes helps improve system reliability and accuracy.

Table 5.6 : Common Failure Modes in RAG Systems

Failure Mode	Explanation
No retrieval	The system cannot find any document matching the user query.
Over-retrieval	Too many documents are retrieved, including irrelevant ones, causing noise.
Contradiction	Retrieved chunks contain conflicting or opposite information.
Citation mismatch	The citation number used in the answer does not correspond to the retrieved source.
Context overflow	The retrieved text exceeds the LLM's token limit, causing truncation or errors.

Mitigation Techniques

To prevent these failures and improve accuracy, RAG systems commonly use:

- **Chunk threshold tuning**
Ignore low similarity scores so only strong matches are used.
- **Top-k reduction**
Retrieve only the top few most relevant chunks (e.g., top-3).
- **Deduplication**
Remove repeated or identical chunks to reduce noise.
- **Short context summaries**
Summarise long documents to fit within LLM token limits.

These techniques help maintain **clean, relevant, and high-quality** retrieval results.

5.9 Case Study: E-commerce Customer Support RAG Bot

Objective

The objective of this case study is to build a simple e-commerce customer support chatbot that answers shopper queries using verified retail information such as return policies, delivery timelines, product specifications, payment rules, warranty terms, and other frequently asked service details.

Steps in the Pipeline

1. **Create an E-commerce FAQ CSV file**
Prepare a dataset containing product-related questions, customer service answers, policies, and category labels such as returns, shipping, payments, and warranty.
2. **Chunk the FAQ entries**
Break each policy or product description into small segments of about 100 to 150 words for efficient retrieval.
3. **Embed the chunks**
Convert each chunk into numerical vectors for similarity matching using cosine similarity.
4. **Retrieve the top three passages**
Identify the three most relevant chunks based on how closely they match the customer's query.

5. Insert the retrieved passages into the RAG prompt template

Supply these chunks to the LLM as contextual evidence for grounded generation.

6. Generate a grounded answer with citations

The LLM produces a verified response that relies only on the retrieved retail data and includes clear citations to indicate the source chunk.

Example Interaction

Customer:

“Can I return an online purchase at a physical store?”

Bot:

“Yes, online purchases can be returned at any of our physical stores within thirty days of delivery. (Source 3)”

This interaction demonstrates how the RAG bot provides a grounded, citation-based answer directly from the verified e-commerce dataset.

5.10. Ethical Compliance in RAG

Ethical practice is essential when developing retrieval-based systems, particularly in commercial environments where customer trust and data safety are critical.

Data Integrity Principles

- Use only information that is publicly available or officially approved for customer support.
- Do not include personal, private, or sensitive customer data in the retrieval corpus.
- Conduct regular reviews of the dataset to identify outdated, incorrect, or risky content and remove it promptly.

Prompt Compliance Principles

- Enforce clear refusal rules such as “No answer found in the documents” whenever required information is missing.
- Ensure every fact in the LLM’s answer is supported by a verified source from the retrieved data.
- Prevent the model from adding any extra information based on its internal training data so that the output remains fully grounded in the retrieved documents.

5.11 Guided Lab Exercise

Pair Assignment

In this lab activity, students will work in pairs to build a mini RAG system using a sample e-commerce FAQ dataset stored in a CSV file.

Each pair must implement the following:

- A retrieval component that uses cosine similarity to identify the most relevant FAQ chunks.
- A grounded generation step where the LLM answers strictly using the retrieved sources.
- A citation tagging method to show which source supports each part of the answer.

Submission Format

Each pair must submit the following elements:

Customer Query
→ Top 3 Retrieved Sources
→ Model Answer
→ Citation Tags (Source numbers)

Example

Query: "Can I return an online purchase at a retail store?"
Top Sources: [Source 3, Source 1, Source 5]
Model Answer: "Yes, online purchases can be returned at any retail store within thirty days.
(Source 3)"

5.12 Classroom Discussion

Debate Topics

The class will analyze and debate the following questions:

1. Can RAG completely eliminate hallucinations, or does it only reduce them?
2. What is the optimal number of retrieved documents to use: top 1, top 3, or top 5?
3. What level of citation detail is necessary for transparency and user trust?

These discussions help students learn the practical decision-making involved in designing effective and reliable RAG systems.

5.13 Module Review Points and Their Key Insights

Review Point	Explanation
RAG integrates retrieval and LLM reasoning	Accuracy improves because the LLM uses verified documents instead of relying on internal memory.
Grounded prompts ensure safe behavior	The model is restricted to retrieved content, preventing guessing or hallucination.
Citations make answers traceable	Users can check where each fact came from, improving transparency and trust.
Threshold rules reduce hallucination	Low-confidence retrieval results are rejected, preventing inaccurate answers.
Ethical safeguards support responsible deployment	Only safe, approved data is used, avoiding privacy risks or misuse of sensitive information.

MODULE 06

EVALUATION: ACCURACY & FAITHFULNESS OF GENAI SYSTEMS

6.1 Introduction and Module Overview

Building Generative AI applications such as chatbots or RAG-based systems is only half the task. An equally important responsibility is evaluating whether the system produces responses that are accurate, helpful, safe, and faithful to source information. Unlike classical machine learning models, which can be evaluated using strict numeric formulas, LLM-based systems require a hybrid evaluation approach combining automatic metrics and human judgment.

This Module introduces learners to basic retrieval accuracy metrics like Precision@K, manual grading rubrics for LLM output quality, and practical methods for testing RAG systems using spreadsheets and structured logs. The objective is to create transparent evaluation workflows suitable for small-scale academic or educational deployments using only free tools.

6.2 What Is Evaluation and Why Is It Essential? (Definition)

Evaluation is the process of checking whether a GenAI or RAG system produces answers that are accurate, consistent, fair, and grounded in the retrieved documents.

It helps identify mistakes, measure system quality, and ensure that the AI output remains trustworthy and aligned with real data.

Why Evaluation Is Essential: Key Reasons

- It reduces hallucinations by verifying answers against facts.
- It improves answer consistency across different queries.
- It ensures higher factual reliability for critical use cases.
- It enables transparent, measurable grading of system performance.

Without proper evaluation, systems may:

- Share or repeat incorrect information
- Display hidden bias in answers
- Fail silently without any warning or detection

Example Scenario

Test Question:

“What is the return period for online purchases?”

RAG System Output:

“You may return online purchases within thirty days. (Source 4)”

Evaluation Outcome:

- **Accuracy:** Correct
- **Faithfulness:** Matches the retrieved policy document
- **Score:** 1.0 (High fidelity)
- **Feedback:** Answer grounded and consistent

Table 6.1: Evaluation Components and Their Purpose

Evaluation Component	Description	Purpose
Test Questions	A structured set of questions representing real user queries.	Helps measure how well the system handles different scenarios.
RAG Pipeline Execution	Retrieval of relevant chunks followed by grounded LLM generation.	Ensures that answers are based on verified documents.
Model Answers	Responses produced by the GenAI system.	Forms the evidence used to judge accuracy and faithfulness.
Metric Calculation	Numerical scoring: accuracy, similarity, citation match, grounding rate.	Provides measurable evidence of system quality.
Human Review	Manual checking of errors, tone, and factual grounding.	Detects subtle mistakes that automated metrics miss.
System Improvement	Adjusting chunking, thresholds, or prompts based on feedback.	Strengthens future responses and reduces errors.

Evaluation Workflow

Evaluation is a crucial part of building reliable GenAI systems. By combining automated metrics with human judgment, organizations can detect errors early, reduce hallucinations, verify citations, and ensure that every answer remains accurate and grounded in real data. Effective evaluation ultimately strengthens the overall performance and trustworthiness of the RAG system.

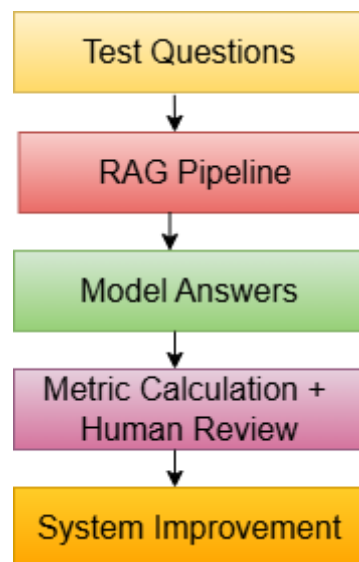


Figure 6.1 : Evaluation Workflow

6.3 Creating Gold Test Datasets

What Are Gold QA Sets? (Definition)

A Gold Test Dataset is a collection of verified question and answer pairs that are manually prepared by subject experts.

These serve as the ground truth against which a RAG system's answers are evaluated.

Example of a Gold QA Dataset

Question	Ground Truth Answer
What time does the library open?	8am to 5pm
What documents are required for hostel admission?	ID proof, admission form, and photographs

Gold Test Sets Help With:

- Providing consistent evaluation benchmarks
- Avoiding subjective or opinion-based grading
- Detecting hallucinations or incorrect responses

Recommended Test Set Size

For student laboratory experiments:

- Minimum size: 8 to 10 questions
- Recommended size: 25 to 30 questions

A larger test set provides better coverage of different question types and improves evaluation quality.

6.4 Precision@K – Retrieval Accuracy Metric

Definition

Precision at K (Precision@K) measures how many of the top K retrieved documents are actually relevant to the question.

It shows how well the retrieval system is selecting useful chunks.

Formula

$$\text{Precision@K} = \frac{\text{Number of Relevant Documents in Top-K}}{K}$$

Example Calculation

A query retrieves the top 3 chunks:

Retrieved Chunk	Relevance
Chunk 1	Relevant
Chunk 2	Relevant
Chunk 3	Irrelevant

$$\text{Precision@3} = \frac{2}{3} = 0.67$$

Why Precision@K Matters

- **Low Precision@K** → The LLM receives irrelevant or noisy context, resulting in weak or incorrect answers.
- **High Precision@K** → Retrieval returns useful information, improving grounding and final answer quality.

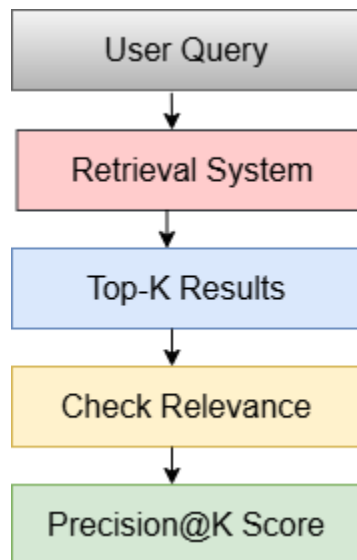


Figure 6.2 : Precision@K Overview

Gold QA datasets and metrics such as Precision at K are essential for evaluating the quality of a RAG system. They help ensure that retrieval produces relevant information and that the model's final responses remain accurate, grounded, and free from hallucinations. Proper evaluation strengthens both reliability and user trust in GenAI systems.

6.5 Faithfulness vs Helpfulness

GenAI responses must be both factually correct and useful to the user.

These two qualities are evaluated using the principles of faithfulness and helpfulness.

Faithfulness

Faithfulness measures whether the answer strictly follows the information found in the retrieved sources.

Example

If the retrieved document does not mention library opening time, but the answer says:

“The library opens at 9am,”

then the output is **unfaithful**, because it introduces new facts not present in the sources.

Faithfulness ensures that answers **do not invent or assume information**.

Helpfulness (Definition)

Helpfulness measures how clearly and completely the answer is written.

This includes:

- Clarity of explanation
- Completeness of the response
- Tone and readability
- How well the answer addresses the user's question

A helpful answer should be easy to understand and directly relevant.

Ideal Output

A high-quality GenAI answer must include:

- **High faithfulness** (grounded in retrieved documents)
- **High helpfulness** (clear, complete, and easy to read)

One quality alone is not enough.

Table 6.2 : Evaluation Matrix for Faithfulness and Helpfulness

Faithful	Helpful	Grade
Yes	Yes	Excellent
Yes	No	Incomplete
No	Yes	Risky hallucination
No	No	Unacceptable

Evaluating faithfulness and helpfulness together provides a complete picture of answer quality in GenAI systems. A response must be grounded in verified documents while also being clear and easy to understand. Balancing both qualities reduces hallucination, improves user trust, and ensures that RAG-based answers remain reliable and meaningful.

6.6 Human Evaluation Rubric

Human evaluation provides a transparent and reliable way to grade GenAI responses. A scorecard helps reviewers assess key qualities such as accuracy, clarity, and safety.

Table 6.3 : Human Evaluation Rubric (Criteria and Score Ranges)

Criterion	Score Range	What It Measures
Faithfulness	0 to 2	Whether the answer is fully supported by retrieved sources.
Relevance	0 to 2	How well the answer matches the user's question.
Clarity	0 to 2	Readability, structure, and ease of understanding.
Safety Compliance	0 to 2	Adherence to refusal rules and avoidance of unsafe content.
Citation Correctness	0 to 2	Accuracy of source references and citation placement.

Total Possible Score: 0 to 10

Table 6.4 : Score Interpretation Guide

Total Score	Interpretation
9–10	Excellent response
7–8	Good response
5–6	Needs improvement
Below 5	Unsatisfactory output

A structured human evaluation rubric ensures that GenAI outputs are reviewed consistently and fairly. By scoring faithfulness, relevance, clarity, safety, and citation accuracy, evaluators can

pinpoint weaknesses and guide system improvements. This approach strengthens the reliability and accountability of RAG-based responses.

6.7 Spreadsheet-Based Evaluation Logs

Definition

A Spreadsheet-Based Evaluation Log is a structured table used to record and assess how well a RAG or GenAI system performs on test questions.

It helps track whether the model answers correctly, stays faithful to the source documents, and retrieves relevant chunks.

Students typically use **Google Sheets** to maintain these logs.

Example Scenario

Imagine evaluating an e-commerce customer support RAG bot.

We ask questions such as:

- “What is the return period for online orders?”
- “What documents are needed for exchanging a product?”

We compare the model’s answers with the **gold (ground truth)** answers and score them.

Table 6.5 : Sample Evaluation Log (E-commerce Dataset)

Query	Gold Answer (Ground Truth)	Model Answer	Faithful? (Yes/No)	Helpful? (Yes/No)	Precision@3	Notes
Return policy	Products may be returned within 30 days	Returns allowed in 30 days	Yes	Yes	1.00	Fully correct
Exchange documents	Original bill and product packaging required	Bill + packaging required	Yes	Yes	0.67	Slight wording difference
Warranty period	Electronics have a 12-month warranty	Warranty is 1 year	Yes	Yes	1.00	Acceptable paraphrasing
Cash on delivery rules	COD available for orders below ₹5000	COD only for orders below ₹5000	Yes	Yes	1.00	Correct match

Benefits of Spreadsheet Evaluation Logs

- **Transparent documentation** of each test question and model output
- **Clear debugging insights** to identify retrieval or prompt failures
- **Tracking improvements** as system changes are tested repeatedly
- **Easy sharing** among team members and instructors
- **Organized structure** for both human and automated evaluation

6.8 Error Analysis

Definition

Error Analysis is the process of identifying and categorizing the types of mistakes a GenAI system makes.

By understanding the error categories, students can apply corrective techniques to improve system performance.

Table 6.6 : Common LLM Error Types

Error Type	Description	Example (E-commerce)
Hallucinated facts	Model adds information not supported by retrieved documents.	Answer claims “90-day return period” when documents say “30 days.”
Partial retrieval	Important policy or detail was not retrieved.	Model misses the “original bill needed” requirement.
Poor explanation	Answer is factually correct but unclear or incomplete.	“Yes, you can return it,” without stating conditions.
Safety failures	Model responds to restricted or sensitive data.	Revealing internal pricing policy or customer records.

Error Correction Techniques

To reduce errors, students can apply the following methods:

- **Improve chunking strategy**
Ensure chunks are meaningful and cover complete ideas.
- **Add stronger prompt guardrails**
Include instructions such as “Use only retrieved sources” or “Do not guess.”
- **Narrow retrieval thresholds**
Only allow chunks with a similarity score above a fixed value.
- **Expand the Gold QA Set**
Include more test questions to catch edge cases.

Spreadsheet-based evaluation logs and systematic error analysis help students understand exactly how their GenAI or RAG system performs. By recording outputs, identifying mistakes, and applying correction techniques, learners can steadily improve both retrieval quality and model faithfulness, resulting in more reliable and accurate AI systems.

6.9 Improving RAG via Feedback Loops

Retail GenAI systems improve over time by using structured feedback loops. These loops refine prompts, improve data quality, and raise the accuracy of customer support responses.

Prompt Optimization Loop

This loop helps retail chatbots improve their wording, instructions, and grounding behavior.

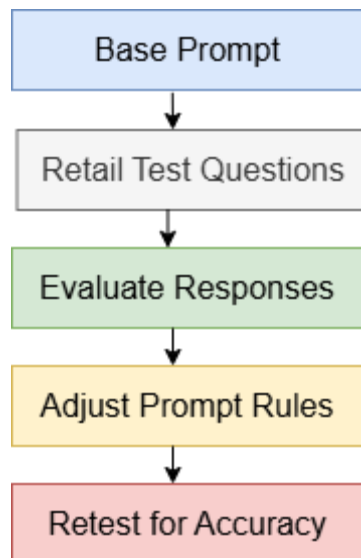


Figure 6.3 : Prompt Optimization Loop

To correct hallucinations, improve clarity, and ensure the bot follows return policies, warranty rules, and product information accurately.

Data Optimization Loop

Better retail datasets lead to stronger retrieval quality.

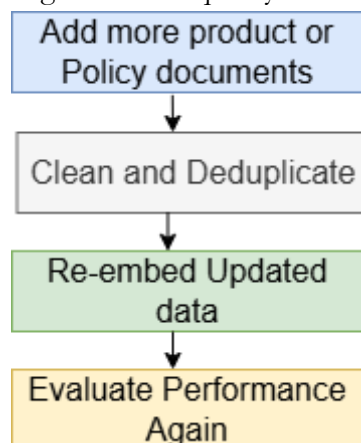


Figure 6.4 : Data Optimization Loop

To remove outdated policies, reduce duplicates, and improve chunk coverage for items like return rules, shipping terms, warranty periods, and payment guidelines.

6.10 Case Study: Improving an E-Commerce Support Bot Initial Evaluation Metrics

Initial Evaluation Metrics

- **Precision@3:** 0.68
- **Faithfulness Score:** 7 out of 10

These results show that the chatbot retrieved several irrelevant chunks and occasionally added extra details not present in the retail documents.

Actions Taken

- Removed near-duplicate product FAQ chunks
- Added refusal rules for queries not covered in the dataset

- Cleaned outdated return and warranty policies
- Improved chunk boundaries for long product descriptions

Improved Evaluation Metrics

- **Precision@3:** 0.87
- **Faithfulness Score:** 9 out of 10

The bot now retrieves more relevant product and policy documents and gives grounded answers with fewer hallucinations.

6.11 Guided Lab Task

Students will evaluate the performance of a **retail RAG system** using a set of sample customer queries.

Students must evaluate:

- **8 test questions** related to returns, exchanges, product details, delivery timelines, etc.
- **Compute Precision@3** for each question
- **Complete the human evaluation rubric** (faithfulness, clarity, safety, citation correctness)

Submission Task Format

Each student submits:

Customer Query → Retrieved Chunks → Precision@3 Score → Error Category

Example:

“Can I return an online purchase at a physical store?”

→ Relevant chunks retrieved

→ Precision@3 = 0.67

→ Error category: Partial retrieval

6.12 Ethics and Evaluation

Evaluation plays a key role in ensuring that retail support bots behave responsibly and safely.

Evaluation Ensures:

- Responsible deployment
- Accurate customer support
- Prevention of harmful or misleading responses
- Fair grading during system testing

Evaluation Should Not Be:

- Subjective
- Biased
- Incomplete or inconsistent

Systematic evaluation protects both customers and retailers.

6.13 Module Review Points

Concept	Explanation
Structured evaluation is essential	Retail GenAI systems must be tested using fixed criteria.
Precision@K measures retrieval quality	Higher precision indicates more relevant product or policy chunks retrieved.
Faithfulness ensures grounding	The bot must answer only from verified retail documents.
Helpfulness measures usability	Answers must be clear, readable, and customer-friendly.
Rubrics support systematic grading	Faithfulness, relevance, and safety are scored consistently.
Feedback loops improve quality	Iterative updates refine prompts, data, and system performance.

Retail RAG systems become more accurate and trustworthy when evaluated through structured feedback loops and human scoring methods. By refining prompts, improving datasets, and using precision metrics, developers can create reliable customer support bots that produce grounded, helpful, and safe responses.

MODULE 07

CHATBOTS: STATE, MEMORY & TOOLS

7.1 Introduction and Module Overview

Modern chatbots extend far beyond basic question–answer pairs. To handle real conversations, bots must remember previous interactions, manage evolving dialogue states, and collect structured information from users through controlled prompts. This Module introduces how chatbots maintain conversation state, store temporary or long-term memory, and safely execute actions using tool integrations.

Students learn that building reliable chatbots involves designing dialogue flows, slot-filling mechanisms, tool safety guardrails, and memory schemas rather than relying solely on prompt-based free conversation. These principles are essential for applications such as campus help desks, admissions assistants, appointment schedulers, and grievance reporting bots.

7.2 Basics of Conversational Agents

2.1 What Is a Chatbot? (Definition)

A chatbot is an artificial intelligence system that interacts with users through natural language. It can hold conversations, answer questions, provide guidance, and complete tasks through text or voice.

Chatbots generally operate in two basic modes based on how they handle conversation context.

Table 7.1: Types of Chatbots and Their Behavior

Type	Description	Memory Handling
Stateless Chatbots	Treat every user query as a new, unrelated input.	No memory of previous messages.
Stateful Chatbots	Understand and maintain the flow of conversation across multiple turns.	Retain context from past messages.

Example: Stateless vs Stateful Chatbot

Stateless Chatbot Example

User: “Store return policy?”

Bot: “Products may be returned within 30 days.”

User (next question): “Is it different for electronics?”

Bot: “Please specify the product.”

(The bot does not remember the topic of returns.)

Stateful Chatbot Example

User: “Store return policy?”

Bot: “Products may be returned within 30 days.”

User: “Is it different for electronics?”

Bot: “Yes, electronics have a shorter return period of 15 days.”

(The bot correctly continues the conversation because it remembers the context.)

Stateless and stateful chatbots behave differently based on how they store conversation history. Stateless bots handle each question separately, while stateful bots maintain context and provide more natural, human-like responses. Understanding these foundations is important for designing effective conversational systems for customer support, retail services, and interactive AI applications.

7.3 Conversation State

Conversation state refers to the information a chatbot stores about what has happened so far in a conversation.

It helps the system understand context and continue the dialogue logically.

A conversation state may include:

- **User intent** (what the user is trying to do)
- **Information already collected**
- **Details still missing or unanswered**
- **The step the chatbot should perform next**

Conversation state is essential for stateful chatbots that maintain context across multiple messages.

State Data Structure

Conversation state is usually stored as a **dictionary, key-value map, or JSON object**, because these structures are simple, flexible, and easy for chatbots to update after each message.

Table 7.2 : Components of a Typical Chatbot State

State Component	Description	Example Value
Intent	The goal or purpose of the user.	"hostel_registration"
Slots	Specific details the chatbot needs to collect.	Name, department, year
Filled Information	Data already provided by the user.	department: "Computer Science"
Missing Information	Data the chatbot still needs.	student_name, year
Current Step	The step the bot is executing next.	"collect_year"

Example: JSON Representation of Conversation State

```
{
  "intent": "hostel_registration",
  "slots": {
    "student_name": null,
    "department": "Computer Science",
    "year": null
  },
  "current_step": "collect_year"
}
```

Explanation of the Example

- The user is performing a **hostel registration** task.
- The chatbot already knows the **department**.
- It still needs **student_name** and **year**.
- The chatbot's next action is to **ask for the year**.

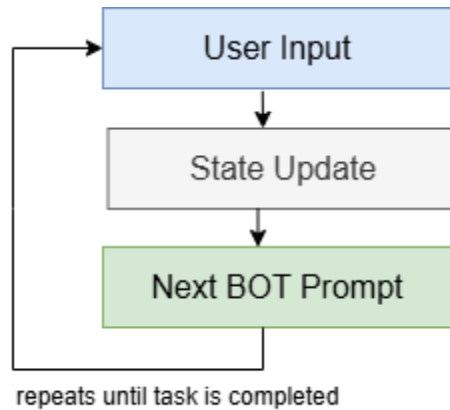


Figure 7.1 : Chatbot State Flow

The chatbot updates the state after every user message and continues the conversation based on what is already known and what is still needed.

Conversation state allows a chatbot to behave intelligently by remembering past inputs and tracking what information is required to complete a task. By storing the user's intent, collected details, and the next step, the chatbot can guide the conversation smoothly and provide accurate, context-aware responses.

7.4 Slot-Filling Dialogue Models

What Is Slot Filling?

Slot filling is a structured dialogue technique used by chatbots to collect all the required pieces of information (called “slots”) from the user before completing a task.

Each slot represents one specific detail the chatbot needs, such as a name, department, product type, date, or location.

The chatbot asks for one slot at a time, updates the conversation state, and stops only after all required information is completed.

Example: Hostel Application Chatbot (Slot-Filling Flow)

Required Slots

- Name
- Department
- Academic Year

Conversation Flow

Bot: What is your name?

User: Anita.

Bot: Which department are you from?

User: Civil Engineering.

Bot: What academic year?

User: 2nd Year.

State completed → **submit form.**

Once all required slots are filled, the chatbot can perform the action such as submitting a form, generating a confirmation message, or storing the information.

Table 7.3 : Slot-Filling Components

Slot Name	Description	Example Value
Name	The user's full name required for identification.	Anita
Department	Academic or organizational department.	Civil Engineering
Academic Year	The current year of study.	2nd Year
Completion State	Indicates whether all required slots are filled.	Completed

Slot Collection Diagram (Workflow)

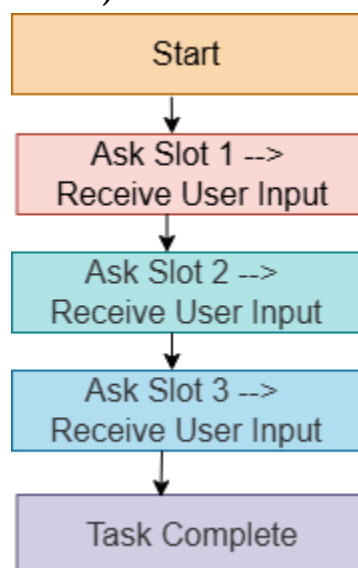


Figure 7.2 : Slot Collection Diagram (Workflow)

The chatbot progresses step by step, collecting information and updating the state after every response.

Slot-filling dialogue models help chatbots complete structured tasks by gathering required information in a systematic order. By collecting each slot one by one and updating the conversation state, the system ensures accuracy and completeness before performing the final action. This approach is widely used in form submissions, registrations, reservations, and customer support workflows.

7.5 Memory Types in Chatbots

Short-Term Memory (STM)

Short-term memory stores temporary information needed only during the current conversation session.

Examples of What STM Stores

- The last question the user asked
- Slots collected so far (name, product, date, etc.)
- Pending form fields
- Current conversation topic

Where STM Is Stored

- Local dictionaries
- In-memory cache
- Session variables

Short-term memory is deleted when the session ends.

Long-Term Memory (LTM)

Long-term memory stores information across multiple sessions.

It helps chatbots personalize future interactions or remember past activity.

Examples of What LTM Stores

- Frequently asked question (FAQ) patterns
- Past chat transcripts
- Non-sensitive user preferences such as preferred language or category
- Saved customer support interactions

Where LTM Is Stored

- Files (JSON, text, or CSV)
- Databases or knowledge bases
- External storage systems

This type of memory requires strict privacy rules because it persists beyond a single session.

Table 7.4 : Short-Term vs Long-Term Memory Comparison

Memory Type	Duration	What It Stores	Example
Short-Term Memory	Only during the active session	Recent messages, collected form fields, conversation flow	Pending slot: “Department: Not collected yet”
Long-Term Memory	Stored across many sessions	FAQ data, language preferences, chat logs	User prefers replies in English

7.6 Tools in AI Systems

Tools are controlled functions or external operations that a chatbot can call to perform specific tasks.

They add **real-world capability** to chatbots beyond simple conversation.

Tools allow the chatbot to:

- Look up a database

- Read or write files
- Perform calculations
- Submit forms
- Retrieve structured information

Table 7.5 : Example Tools and Their Purpose

Tool Name	Purpose
get_timetable()	Retrieves class or meeting schedules
submit_application()	Saves or submits a filled form
calculate_fees()	Computes total fee amount or charges
get_order_status()	Retrieves order tracking information
check_stock()	Checks product availability

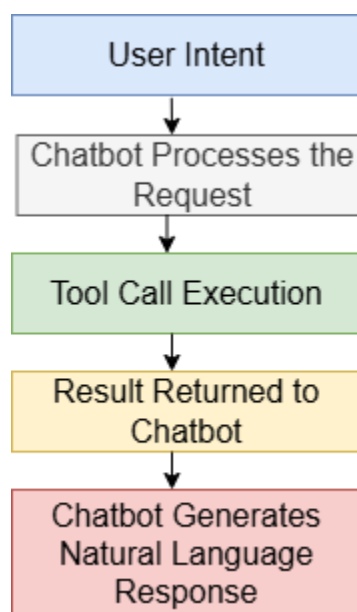


Figure 7.3 : Tool Invocation Cycle

Example:

User asks, “How much is the total fee?”

→ Chatbot calls `calculate_fees()`

→ Receives result

→ Replies: “Your total fee is 5,200 rupees.”

Short-term and long-term memory allow chatbots to maintain context and personalize interactions, while tools give chatbots the power to perform real tasks such as database lookups and calculations. Together, memory and tools form the foundation of intelligent, efficient, and context-aware conversational agents that can support a wide range of real-world applications.

7.7 Tool Guardrails & Safety Controls

Tool guardrails are safety rules that decide **when and how** a chatbot is allowed to call tools such as database functions, file access, or calculation routines.

They protect users from harmful actions, privacy leaks, and unsafe operations.

Tools must **not** be executed freely. They should only be used when specific **safety conditions** are met.

Safety Requirements for Tool Use

Before calling a tool, the system should check:

- **Slot values are validated**
Ensure inputs like dates, amounts, or IDs are in correct format.
- **Sensitive information is blocked**
Requests that involve confidential or personal data must be rejected.
- **Input sanitization**
Remove or neutralize harmful content such as code injection or malicious text.
- **Execution logs maintained**
Record tool calls and responses for later auditing and debugging.

Table 7.6: Tool Safety Requirements and Their Purpose

Safety Requirement	Description	Purpose
Slot validation	Check that all required fields are present and valid.	Prevents incorrect or incomplete operations.
Sensitive info blocking	Detect and reject private or confidential queries.	Protects user privacy and security.
Input sanitization	Clean up user inputs before tool execution.	Avoids malicious or unintended commands.
Execution logging	Record tool usage and responses.	Supports auditing, debugging, and compliance.

Unsafe Example

Query:

“Withdraw money from account number 123456.”

Problem:

- This involves personal finance and account access.
- It violates privacy and security policies.
- A safe chatbot must never perform such operations via tools.

Correct behavior:

Reject this request with a safe message such as:

“I am not allowed to access or modify bank accounts.”

Safe Tool Prompt Rule

A common guardrail rule is:

Use tools only for publicly authorized queries.

Reject requests that involve personal finance, health, or identity data.

This ensures tools are used only for safe operations such as:

- Retrieving public FAQs

- Checking generic schedule information
- Calculating simple totals (without real account data)

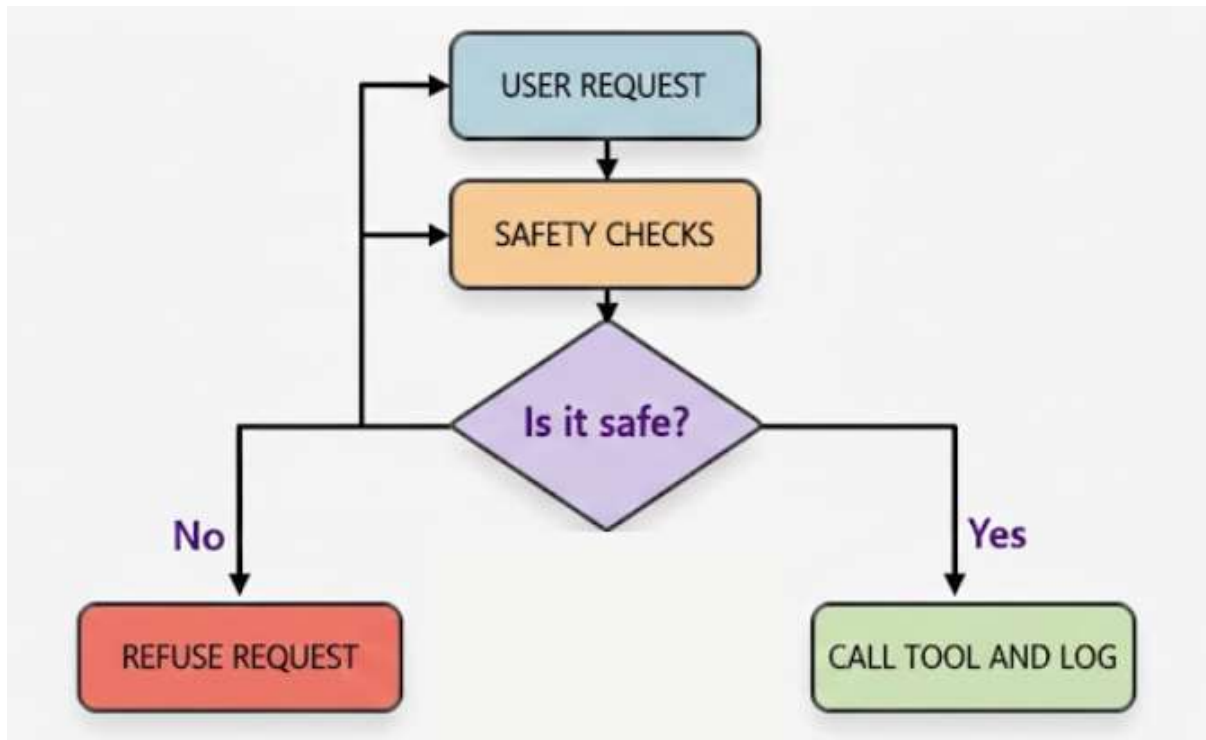


Figure 7.4 : Tool Guardrail Flow

7.8 Prompt Chaining in Chatbots

Prompt chaining is a method where multiple prompts are connected in a sequence to form a structured workflow.

Instead of one big prompt, the chatbot passes outputs from one step to the next step.

This is useful for complex tasks that require several stages, such as extracting information, retrieving documents, and generating a final answer.

Prompt Chaining Workflow

A typical prompt chaining pipeline may look like:

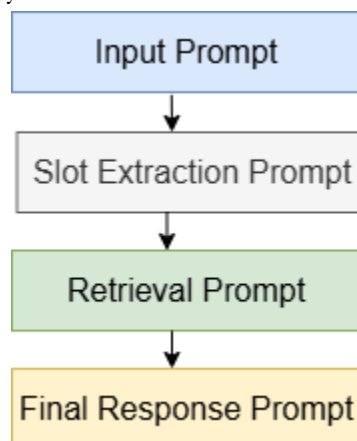


Figure 7.5 : Prompt Chaining Workflow

Step-by-step Explanation

1. Input Prompt

The user message is received and understood.

2. Slot Extraction Prompt

A prompt is used to extract required details, such as product name, date, or order ID.

3. Retrieval Prompt

Another prompt is used to search documents, FAQs, or databases using the extracted information.

4. Final Response Prompt

The final prompt asks the LLM to generate a clear and grounded natural language response based only on the retrieved content.

Table 7.7 : Stages in a Prompt Chaining Workflow

Stage	Prompt Role	Example Function
Input Prompt	Capture the user query.	“What is your question?”
Slot Extraction Prompt	Identify key details from the query.	Extract “product name” and “issue type.”
Retrieval Prompt	Retrieve relevant documents or records.	Search FAQ for product return rules.
Final Response Prompt	Generate the final answer.	Explain return policy in simple language.

Benefits of Prompt Chaining

Prompt chaining:

- **Controls logic branching**

Each step has a specific role and decision logic.

- **Enables step-wise reasoning**

The system solves smaller tasks in sequence rather than everything at once.

- **Reduces hallucinations**

The final answer is based on structured intermediate steps and retrieved information, not free-form guessing.

Tool guardrails and prompt chaining are essential design patterns for safe and reliable chatbots. Guardrails ensure that tools are called only in secure, controlled situations, while prompt chaining breaks complex tasks into clear, manageable steps. Together, they improve answer quality, reduce hallucinations, and protect users from unsafe or unauthorized actions.

7.9 Teaching Case — Retail Support Bot Conversation Flow

Scenario

A retail chatbot helps customers complete a **product return request** using a slot-filling conversation flow.

Conversation Example

Customer: “I want to return a product.”

Bot: “May I have your order ID?”

Customer: “OD14582.”

Bot: “What is the reason for the return?”

Customer: “The product is damaged.”

Bot: “When was the product delivered?”

Customer: “Two days ago.”

Result:

All required return-related slots are now collected, and the chatbot can submit the return request.

State Snapshot (JSON Example)

```
{
  "intent": "product_return",
  "slots": {
    "order_id": "OD14582",
    "return_reason": "Damaged product",
    "delivery_date": "2 days ago"
  },
  "status": "complete"
}
```

This JSON object shows what the chatbot currently knows and confirms that all required fields have been collected.

Table 7.8 : Slots Required for a Retail Return Bot

Slot Name	Purpose	Example Value
Order ID	Identify the customer’s order.	OD14582
Return Reason	Understand why the customer wants a return.	Damaged product
Delivery Date	Validate return eligibility based on timeline.	Two days ago
Status	Indicates whether the return form is complete.	complete

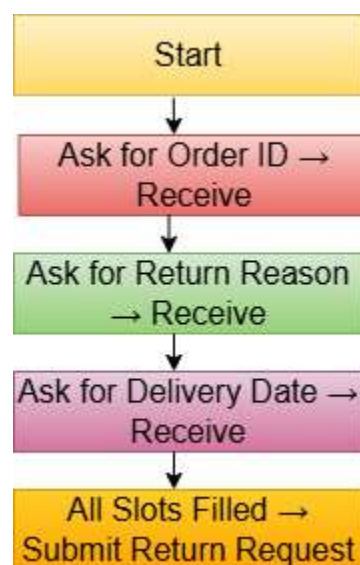


Figure 7.6 : Slot-Filling Conversation Flow

7.10 Ethical Handling of Chat Memory

Chatbots that store user information must follow strict ethical rules to ensure privacy, safety, and compliance.

Retail chatbots especially must protect customer data such as order details and communication history.

Table 7.9 : Privacy Principles for Chat Memory

Rule	Explanation
No sensitive personal data	Do not store identity numbers, bank details, passwords, or private financial information.
Inform users of memory usage	Users should know what data is remembered and why.
Clear deletion policy	Old logs must be periodically deleted or anonymized.
Minimal retention	Store only essential non-sensitive information, such as preferred language or frequent product categories.

Important Ethical Warning

Archiving or training an AI model on **private customer chat logs without user consent** is ethically and legally unacceptable.

Retail systems must avoid storing anything that could identify or harm the customer.

A retail support chatbot must manage conversation state responsibly while collecting only the information needed to complete a task such as a product return. Ethical memory handling ensures user trust and prevents misuse of sensitive data. By combining structured slot-filling and strict privacy rules, chatbots become both effective and safe to deploy in real business environments.

7.11 Guided Classroom Exercise

A **guided classroom exercise** helps students practice building a simple chatbot by designing conversation flow, slot structures, and state updates.

It reinforces concepts such as **slot filling, state tracking, and dialogue control**.

Exercise Task

Objective:

In pairs, students will design a **3-turn chatbot** for a **Campus Lost-and-Found Assistant**.

Required Slots:

The chatbot must collect the following information:

- **Item lost**
- **Location where it was lost**
- **Date it was lost**

Example Conversation Flow

Bot: What item did you lose?

User: I lost my ID card.

Bot: Where did you lose it?

User: Near the library steps.

Bot: When did you lose it?

User: Yesterday.

Once all slots are collected, the chatbot completes the report.

Table 7.10 : Required Slots for Lost-and-Found Assistant

Slot Name	Purpose	Example Value
Item lost	Identify what object is missing	ID card
Location	Know where the loss occurred	Library steps
Date lost	Know when it was lost	Yesterday

Students Must Produce

1. Slot Flow Diagram

Illustrates how the bot collects each slot one by one.

2. State Schema

A JSON or dictionary representing the stored values.

Example State Schema

```
{
  "intent": "lost_and_found_report",
  "slots": {
    "item": "ID card",
    "location": "Library steps",
    "date_lost": "Yesterday"
  },
  "status": "complete"
}
```

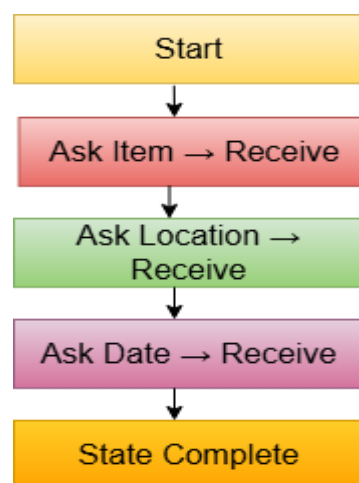


Figure 7.7 : Slot-Filling Flow

7.12 Homework Activity

Slot Validation Assignment

Students must add a validation rule to one slot in their chatbot design. Validation ensures the user enters data in the correct format.

Example: Email Slot Validation

Rule:

The email must match the following regular expression:

```
^[a-zA-Z0-9._%+-]+@[a-zA-Z]+\.[a-zA-Z]{2,}$
```

This ensures the chatbot only accepts properly formatted email addresses.

Table 7.11 : Slot Validation Example

Slot	Validation Rule	Reason
Email	Must match regex pattern	Prevents invalid or unusable email entries

7.13 Module Review Points

Concept	Explanation
Chatbots require state tracking	Context is needed to maintain meaningful multi-turn interactions.
Slot filling enables structured data collection	Ensures tasks like forms or reports are completed step-by-step.
Memory supports multi-turn conversations	Short-term memory retains user responses during the session.
Tools execute actions safely	Guardrails prevent unsafe or unauthorized tool calls.
Ethical data storage is mandatory	Protects user privacy and ensures compliance with data policies.

MODULE 8

NLP TASKS WITH LARGE LANGUAGE MODELS — PRACTICAL PROMPT PATTERNS

8.1 Introduction and Module Overview

Traditional Natural Language Processing focused mainly on single-purpose classifiers such as sentiment analysis or keyword extraction systems trained on labeled datasets. Large Language Models transform NLP workflows by enabling multi-task performance through natural-language instructions rather than model retraining. A single LLM can perform summarization, classification, extraction, rewriting, formatting, and redaction when guided by correctly designed prompts.

This Module introduces students to commonly required NLP tasks and demonstrates how each task can be solved using standardized prompt patterns. Emphasis is placed on structured outputs such as JSON and CSV, which allow seamless use of LLM results in downstream pipelines like databases, retrieval evaluation sheets, or automation scripts.

8.2 Mapping NLP Tasks to Prompt Patterns

Large Language Models (LLMs) can perform multiple Natural Language Processing (NLP) tasks by changing **how the prompt is written**, rather than by training separate machine learning models.

These reusable instructions are called **prompt patterns**.

By selecting an appropriate prompt pattern, the same LLM can summarize text, classify content, extract information, or rewrite text efficiently.

Core NLP Task Patterns Supported by LLMs

LLM-based applications commonly rely on four primary NLP task patterns, as shown below.

Table 8.1: NLP Task Patterns and Their Applications

NLP Task Pattern	Description	Example Use Case
Summarization	Condenses long text into a shorter, meaningful version.	Creating highlights from reports, articles, or policies.
Classification	Assigns text to predefined categories or labels.	Sentiment analysis, topic tagging, spam detection.
Information Extraction	Identifies and pulls key facts into structured formats.	Extracting names, dates, prices, or order IDs into tables.
Text Rewriting	Modifies text without changing its meaning.	Grammar correction, tone adjustment, simplification.

Why Prompt Patterns Are Important

- One LLM can perform **multiple NLP tasks** using different prompts
- No need to train and maintain separate models for each task
- Faster development and easier experimentation
- Suitable for real-world applications such as customer support, analytics, and content management

This flexibility significantly reduces system complexity and development cost.

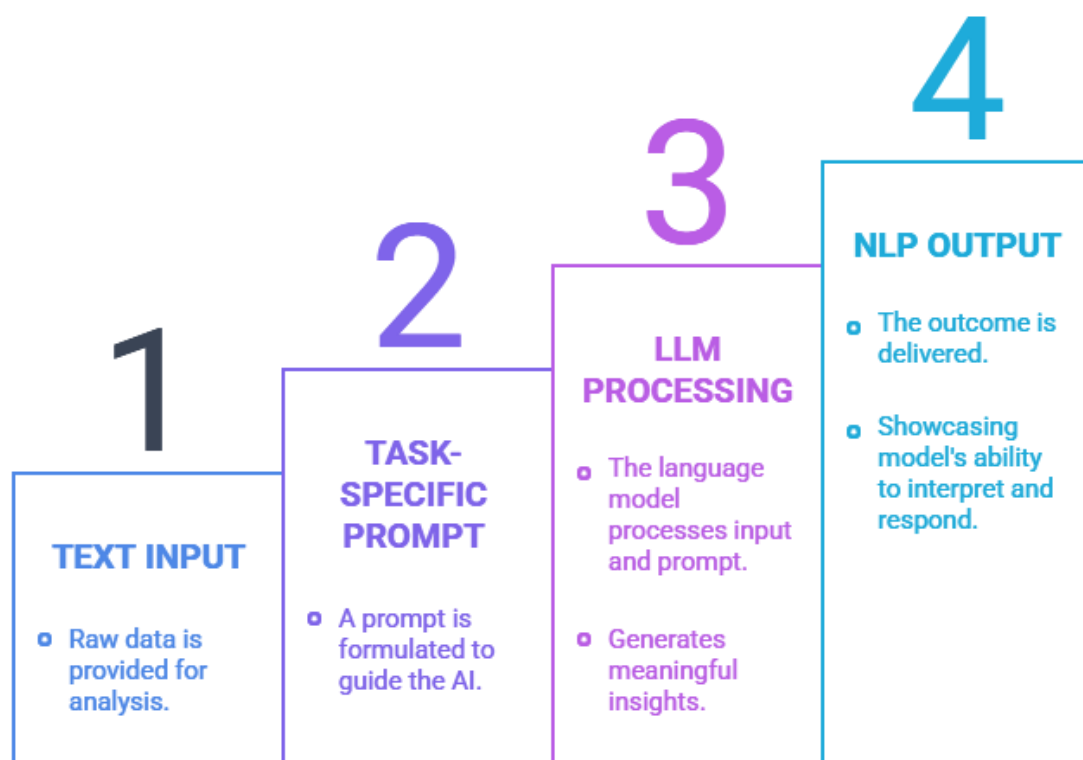


Figure 8.1 : Pattern Diagram — NLP Task Mapping

8.3 Summarization Tasks

What is Summarization?

Summarization is an NLP task where a Large Language Model (LLM) transforms a long piece of text into a concise, meaningful version while retaining the core ideas. It reduces reading time, improves comprehension, and supports information retrieval without altering the original intent of the content.

Types of Summaries

The two major approaches used by LLMs are Extractive and Abstractive.

Table 8.2 : Types of Summaries and Their Characteristics

Type of Summary	Definition	How It Works	Typical Use Cases
Extractive	Selecting key sentences or phrases directly from the source text.	No rewriting, only extraction of the most relevant lines.	News highlights, legal text extraction, meeting minutes.
Abstractive	Creating a rewritten, compressed version using new words.	The model understands, interprets, and paraphrases content.	Academic summaries, concept notes, research briefs.

Standard Prompt Pattern

A commonly used structured prompt for summarization:

Prompt Template:

Summarize the following text into 5 bullet points.
Include only factual information.

Text:
[document]

This prompt enforces:

- Controlled length
- Factual accuracy
- Bullet-point structure

Academic Example

Input:
"This university offers Amrita Merit Scholarships for eligible students of undergraduate programs..."

Model Output:

- Offers merit-based scholarships
- Applies to undergraduate students
- Selection based on academic scores
- Scholarship limits vary
- Application deadline applies

Summarization tasks enable LLMs to convert lengthy documents into concise, meaningful insights, supporting faster decision-making and better comprehension. Whether using extractive or abstractive approaches, well-designed prompt patterns ensure high-quality summaries that remain faithful to the original content.

8.4 Classification Tasks

Text Classification

Text classification is an NLP process where a Large Language Model (LLM) automatically assigns a given text to one of several predefined categories. It is widely used in helpdesks, academic institutions, customer service, sentiment analysis, and automated routing systems. The goal is to interpret the intent of the text and map it to the most appropriate label with high accuracy.

Table 8.3 : Common Classification Categories

Category Type	Example Labels	Purpose / Use Cases
Intent Categories	Complaint, Inquiry, Praise	Routes user messages to correct department or support workflow.
Academic Queries	Admission-related, Hostel-related, Exam-related	Helps educational institutions organize student service requests.
Sentiment	Positive, Neutral, Negative	Used for monitoring student feedback, service quality, and user experience.

Standard Classification Prompt Pattern

Prompt Template:

Classify the following message into one category:

["Complaint", "Inquiry", "Feedback"]

Message:

[item_text]

Output only the category label.

This pattern ensures:

- No explanations
- No extra text
- Single-label output

Example

Input:

“The application website is not loading.”

Output:

Complaint

Classification tasks help LLMs quickly identify the intent or sentiment of a message and place it in the correct predefined category. With structured prompts, the system becomes reliable for institutional workflows such as student services, ticketing, or feedback analysis, ensuring efficient routing and accurate interpretation of user messages.

8.5 Information Extraction Tasks

Extraction Overview

Information extraction is the NLP process of converting unstructured free text into structured data. LLMs identify key facts, entities, and attributes within the text and map them to predefined fields. This makes data actionable for automation, analytics, and system integration.

JSON Extraction Prompt Pattern

Prompt Template:

Extract the following fields:

department

event

date

Output valid JSON only:

Text:

[paragraph]

This pattern instructs the LLM to output structured data without additional text. JSON is chosen because it is both machine-friendly and easy to integrate into applications.

Example

Input Text:

"The Biology Department will conduct orientation on July 12."

Output:

```
{
  "department": "Biology",
  "event": "orientation",
  "date": "July 12"
}
```

Table 8.4 : Importance of JSON

Benefit	Explanation
Machine Readability	JSON can be directly processed by computer programs, APIs, and databases.
Spreadsheet Integration	JSON fields can be imported into Excel, Google Sheets, and dashboards.
Automated Validation	Systems can verify whether values exist, match patterns, or trigger workflows.

Error Handling Rule

A robust extraction prompt includes fallback instructions:

Rule:

If any field is unavailable in the text, assign its value as null.

This ensures:

- No missing keys
- Consistent record structure
- Smooth downstream processing

Information extraction transforms natural language text into structured JSON, enabling automation, data analysis, and seamless integration with digital systems. By defining clear fields and error-handling rules, LLMs can reliably extract consistent records that support institutional workflows, reporting, and intelligent applications.

8.6 Text Rewriting Tasks

Rewrite Operations

Text rewriting is an NLP technique where an LLM transforms a given sentence or paragraph while keeping the original meaning intact. It is used to improve clarity, adjust tone, correct grammar, or adapt language for different audiences. Rewriting helps refine communication without altering the underlying message.

Table 8.5 : Common Rewrite Operations

Rewrite Type	Definition	Purpose / Use Cases
Grammar Correction	Fixes spelling, punctuation, and sentence structure.	Academic submissions, official communication, notices.

Tone Adjustment	Changes the style to formal, friendly, polite, or assertive.	Emails, service messages, support replies.
Simplification	Rewrites text using easy, accessible language.	School-level materials, IEC content, public notices.

Table 8.6 : Rewrite Patterns

Use Case	Prompt Pattern	Expected Output
Simplify	Rewrite at Grade 6 reading level.	Simple, easy-to-read version.
Formalize	Convert to academic tone.	Formal, structured rewrite.
Friendly Tone	Rewrite as polite message.	Warm, courteous version suitable for public communication.

Rewrite Prompt Example

Prompt Template:

Rewrite the following notice in friendly tone while keeping meaning unchanged.

Text:

Original text here.

This ensures:

- Meaning remains the same
- Tone becomes softer and user-friendly
- No extra explanation or summary is added

Text rewriting enables LLMs to refine communication by correcting grammar, adjusting tone, and simplifying complex content. With structured rewrite patterns, users can consistently generate messages tailored for academic, professional, or public contexts while preserving the accuracy and purpose of the original text.

8.7 Structured Output Formats

Structured output formats are essential for automated workflows, analytics systems, dashboards, and machine-readable pipelines. LLMs can generate outputs in formats such as CSV, JSON, XML, or standardized tables. Clear formatting ensures consistency and prevents downstream processing errors.

CSV Generation Pattern

Prompt Template:

Convert the summary points into CSV rows with columns:
"Point_Number", "Description".

This pattern ensures:

- Ordered rows
- Clean column headers
- File-ready CSV output

Table 8.7 : Example of Structured Output Format (CSV Representation)

Point_Number	Description
1	Merit scholarships available
2	For UG students
3	Based on academic scores

8.8 Privacy-Safe Redaction

Redaction is the process of removing sensitive personal information from text before storing, displaying, or analyzing it. LLM-based redaction ensures privacy compliance and prevents identifiable data leaks.

Redaction Prompt Pattern

Prompt Template:

Redact all personal names,
phone numbers,
email IDs from the text.

Replace each with [REDACTED].

This allows:

- Clean anonymized outputs
- Compliance with privacy laws
- Safe use of text in training or analytics

Redaction Example

Before:

"Anita's phone is 9876543210."

After:

"[REDACTED]'s phone is [REDACTED]."

8.9 Pipeline Pattern — Multi-Task NLP

LLMs support multi-stage workflows by chaining tasks together. A single input can move through summarization, extraction, and formatting to produce structured outputs.

All these tasks can be chained together:

Each stage transforms the text:

- Summarization condenses information
- Extraction identifies key fields
- Formatting produces CSV/JSON for automation

Structured formats like CSV and JSON help LLM outputs integrate smoothly with data pipelines, dashboards, and applications. Combining summarization, extraction, and redaction into a multi-task pipeline allows institutions to automate workflows while maintaining accuracy, privacy, and compliance.

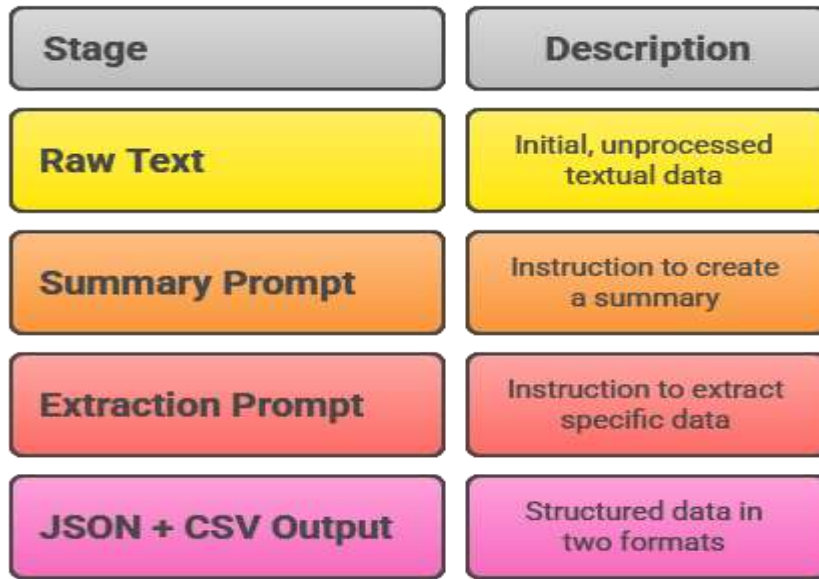


Figure 8.2 : Prompt Pipeline Diagram

8.10 Practical Case Study — Customer Feedback Processing

A retail organization receives nearly **500 customer feedback messages** every week, covering complaints, product queries, service appreciation, and operational concerns. LLM-based NLP enables automated sorting, extraction, summarization, and reporting of these messages, significantly reducing manual workload.

Table 8.8 : Workflow Using LLM NLP Tasks

Step	Task	LLM Operation	Example Output
1	Classify feedback	Complaint / Praise / Inquiry	<i>Complaint</i>
2	Extract key topics	Delivery / Product Quality / Billing / Store Experience	<i>Delivery</i>
3	Summarize complaints	Short, factual summary	“Delayed delivery reported by customer”
4	Generate CSV reports	Weekly operational summaries	Structured dashboard-ready file

Output Example

Message_ID	Category	Topic	Summary
001	Complaint	Delivery	Package delivered late

This format enables customer service teams to quickly identify high-frequency issues such as delivery delays, defective products, or billing errors.

8.11 Guided Lab Exercise

Solo Task for Learners

Participants will process a set of anonymized **customer feedback messages** using LLM-based prompts to convert unstructured text into structured datasets.

Tasks to Perform

- Apply **classification prompts** to categorize each message.
- Execute **information extraction** to pull fields such as topic, date, product, or issue.
- Convert extracted fields into **valid JSON**.
- Store JSON results in a **spreadsheet** for further analysis.

Submission Format

Original feedback text →

LLM prompts used →

Structured JSON result →

Spreadsheet entry

8.12 Ethics of NLP Automation

When processing customer feedback using automated NLP, organizations must ensure:

- **Privacy protection:** Remove personal identifiers such as names, emails, and phone numbers.
- **No mislabeling:** Incorrect categorization may escalate issues wrongly.
- **Correction rights:** Customers should have the ability to dispute or clarify processed information.
- **Human oversight:** AI-generated results must be reviewed before major decisions.

Ethical alignment builds customer trust and prevents reputational risks.

8.13 Module Review Points

- LLMs enable sequential **multi-task NLP** through structured prompts.
- **Summarization** condenses detailed feedback into short, actionable insights.
- **Classification** identifies the message intent (complaint, praise, inquiry).
- **Information extraction** converts raw text into structured fields.
- **JSON output** supports automation and downstream pipelines.
- **Ethical redaction** ensures privacy and compliance with data-protection standards.

MODULE 09

DATA PREPARATION & QUALITY FOR GENERATIVE AI APPLICATIONS

9.1 Introduction and Module Overview

The effectiveness of any AI or NLP system is fundamentally determined by the quality of the data that powers it. In retrieval-based pipelines and evaluation workflows, poor-quality datasets lead to unreliable answers, hallucinations, bias amplification, and inaccurate performance metrics. For small-scale academic demonstrations and free-tier deployments, data quality control becomes even more critical since large volumes of automatically cleaned data are not available.

This Module introduces essential methods for creating clean, usable datasets for RAG systems and evaluation testing, including data sampling strategies, deduplication techniques, document provenance recording, and leakage prevention. Learners are trained to implement ethical and transparent data curation processes using simple tools such as spreadsheets and Python libraries.

9.2 The Data Lifecycle in RAG Systems

Retrieval-Augmented Generation (RAG) systems depend on a well-organized data lifecycle to ensure that retrieved knowledge is accurate, safe, and useful. The data lifecycle refers to the complete journey of information—from collection to preprocessing, indexing, retrieval, generation, and evaluation.

Key Data Stages

The major stages of a RAG data lifecycle typically include:

Table 9.1 : RAG data lifecycle

Stage	Definition	Activity
Data Collection	Gathering documents from approved and verified sources.	Downloading university notifications and handbooks.
Preprocessing	Cleaning, structuring, and normalizing the collected text.	Removing HTML tags, correcting encoding errors.
Indexing	Converting text into embeddings for fast retrieval.	Storing vector embeddings in a vector database.
Retrieval	Fetching the most relevant chunks based on user queries.	Querying “Exam schedule” returns top related paragraphs.
Generation	LLM combines retrieved text with reasoning to produce answers.	Producing a response summarizing exam rules.
Evaluation & Monitoring	Assessing system accuracy, fairness, and safety.	Auditing chatbot responses for errors or bias.

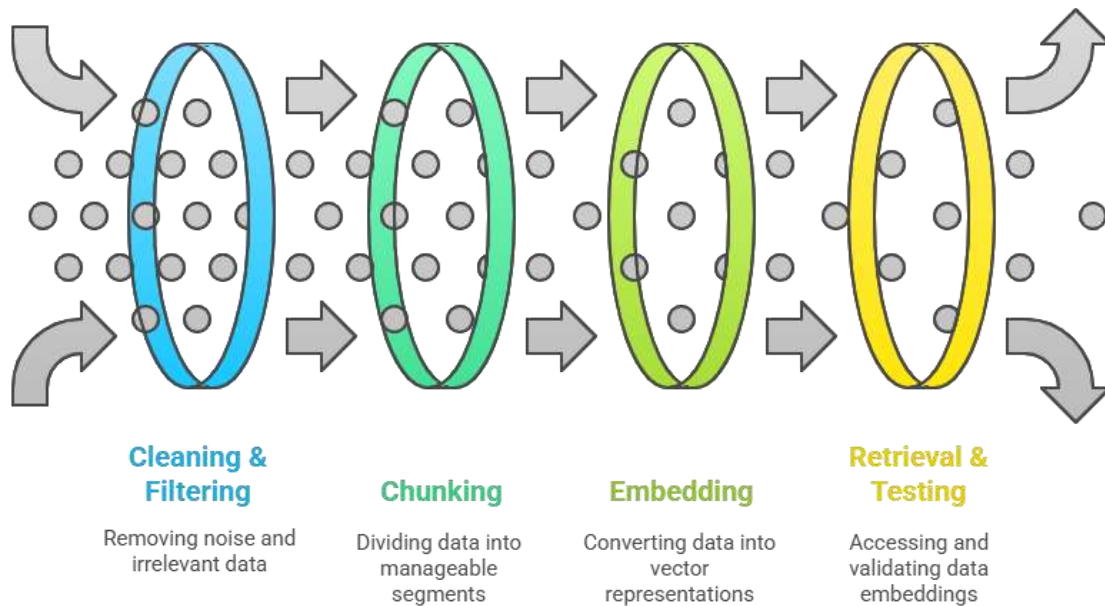


Figure 9.1 : Data Processing in RAG Systems

Lifecycle Objectives

Each stage is designed to maintain:

- Accuracy – Ensuring retrieved content is correct.
- Coverage – Representing all relevant topics.
- Fairness – Avoiding skewed or biased data sources.
- Transparency – Traceability of sources and preprocessing steps.
- Safety – Preventing harmful or private data exposure.

9.3 Data Sources & Collection

Allowed Sources

For educational and public-service chatbots, only publicly available, institution-approved documents must be used. Examples include:

- University notifications
- Prospectus information
- Public handbooks
- Government policy releases

These sources ensure transparency, legality, and accuracy.

Forbidden Data Types

Some data categories must never be included in RAG datasets due to privacy, legal, or ethical risks.

Table 9.2 : Restricted Data and Reasons

Restricted Data	Reason for Restriction
Student grade sheets	Violates personal privacy.
Aadhaar or ID scans	High identity theft risk.
Medical records	Legally protected sensitive data.
Private email messages	Requires consent; copyright concerns.

Golden Rule

If the data cannot be safely displayed to the public, it must not be used in a dataset.

9.4 Sampling & Dataset Construction

Sampling is the process of selecting a meaningful and representative subset of text data from a larger corpus. Well-designed sampling helps create efficient datasets that maintain coverage without unnecessary duplication.

Benefits of Sampling

- Ensures balanced topic representation
- Removes redundant or repetitive content
- Reduces dataset size and improves system efficiency

Stratified Sampling

Stratified sampling divides the dataset into content categories (strata) to ensure equal representation across topics.

Table 9.3 : Example Dataset Construction

Corpus Category	Target Records
Admissions	250
Hostels	250
Library	250
Examinations	250
Total Dataset	1000 Records

This prevents the chatbot from overfitting to a single topic.

Document Diversity

Each category should contain multiple document types to avoid stylistic or structural bias.

Recommended document formats:

- Short notices
- Policy extracts
- FAQ sections

A diverse document mix helps the RAG system generalize better.

A well-structured data lifecycle ensures that RAG systems operate safely, accurately, and transparently. By using only approved sources, avoiding restricted data, applying balanced sampling, and ensuring document diversity, organizations can build reliable datasets that support fair, secure, and effective retrieval-augmented AI models.

9.5 Deduplication Techniques

Deduplication ensures that datasets used in Retrieval-Augmented Generation (RAG) systems remain clean, balanced, and free from repeated content. Removing duplicates prevents inflated performance metrics and improves the reliability of retrieval workflows.

Why Deduplication is Critical

Duplicate or near-duplicate text entries negatively impact model training and evaluation.

Table 9.4 : Effects of Duplicate Records

Problem	Explanation
False performance inflation	Repeated items make evaluation appear better than it actually is.
Over-representation of topics	Some categories become unintentionally dominant.
Biased retrieval results	Search results may repeatedly show the same content.

Manual Deduplication

For small- to medium-sized datasets, spreadsheet tools can be used.

Manual Methods

- **Exact match filtering** — Removes identical rows.
- **Keyword similarity checks** — Flags repeated sentences with slight variations.

Example: Using Excel’s *Remove Duplicates* or conditional formatting to highlight similar entries.

Automated Near-Duplicate Detection

For larger datasets, automated text similarity algorithms help detect subtle variations.

Table 9.5 : Common Techniques

Method	Definition	Use Case
Cosine Similarity	Measures vector similarity between two text embeddings.	Detecting rewritten but identical messages.
TF-IDF Vector Comparison	weighs term importance to detect similarity.	Identifying repeated FAQ answers.
Embedding-based similarity (LLM/transformers)	Uses modern embeddings for semantic matching.	High-quality duplicate detection in RAG pipelines.

Python example:

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

if cosine_similarity(v1, v2) > 0.90:
    print("Near duplicate detected.")
```

Deduplication Rule of Thumb

Similarity $\geq 0.90 \Rightarrow$ treat as duplicate.

9.6 Leakage Prevention

Leakage occurs when the training and evaluation sets overlap, leading to unrealistically high accuracy. Preventing leakage ensures a fair and realistic assessment of RAG system performance.

What is Leakage?

Leakage refers to mistakenly using the same or similar content in both training and evaluation sets.

Real Example of Leakage

- Using the *Admission FAQ* in the training dataset
- And using the same FAQ again as evaluation questions

This causes an **inflated metric**, because the system “already saw the answer.”

Prevention Strategy: Split First, Train Later

Example of Leakage

- Using admission FAQ in both training set and evaluation QAs without separation.

Prevention Strategy

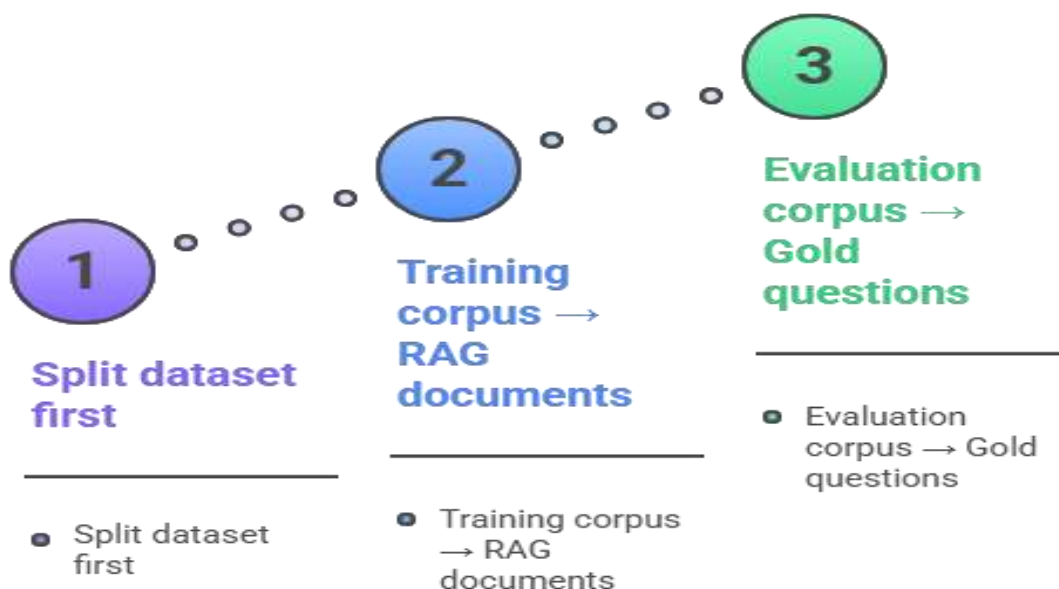


Figure 9.2 : Prevention Strategy: Split First, Train Later

Table 9.6: Recommended Split Ratios

Section	Data Share
Retrieval corpus	80%
Evaluation corpus	20%

This ensures clean separation between what the model learns and what it is tested on.

9.7 Dataset Documentation (Data Cards)

A Data Card provides standardized documentation explaining how a dataset was created, its purpose, risks, and limitations. It enhances transparency and accountability in RAG systems.

What Is a Data Card?

A data card formally describes the dataset's composition, source, and ethical risks. It allows reviewers, developers, and auditors to understand what content the model was exposed to.

Table 9.7 : Mandatory Fields in a Data Card

Field	Description
Purpose	Why the data is collected; intended use.
Source	Document origin and verification details.
Categories	Types of content included (Admissions, Hostel, etc.).
Privacy Assessment	Risks identified and mitigation applied.
Bias Notes	Any representational or language imbalance.

Table 9.8 : Example Data Card

Attribute	Details
Dataset Name	Campus_Faq_2025
Source	Official university handbook
Records	1,000
Categories	Admissions, Hostel, Library, Exam Rules
Privacy	No personal data used
Bias Notes	English-language dominance

Deduplication, leakage prevention, and dataset documentation are essential components of responsible RAG development. By removing duplicates, maintaining strict training–evaluation separation, and creating transparent data cards, organizations ensure that their AI systems remain reliable, fair, and safe to deploy across educational and public-service applications.

9.8 Cleaning & Preprocessing Tasks

Cleaning and preprocessing are essential steps in preparing high-quality datasets for Retrieval-Augmented Generation (RAG) systems. This stage removes noise, enforces consistency, and ensures that no sensitive information enters the training corpus. Clean data directly improves retrieval accuracy and model reliability.

8.1 What is Cleaning & Preprocessing?

It is the systematic process of transforming raw text into a standardized, safe, and machine-readable format.

Table 9.9 : Key Cleaning Steps

Step	Description	Example
1. Remove HTML or artifacts	Deletes tags, symbols, or formatting noise.	 Hostel Notice</br> → Hostel Notice
2. Standardize capitalization	Applies consistent upper/lowercase rules.	“aDmIsSiOn” → “Admission”
3. Remove sensitive identifiers	Deletes emails, phone numbers, names.	john123@campus.edu → [REDACTED]
4. Normalize punctuation	Fixes spacing, quotes, and repeated symbols.	Exam!! Schedule → Exam schedule
5. Verify dates & terminology	Ensures date formats and terms follow a standard.	12/7/25 → 12 July 2025

Example Cleaning Rule

Before Cleaning	After Cleaning
EMAIL: library@campus.edu	EMAIL: [REDACTED]

This illustrates automated privacy-safe redaction during preprocessing.

9.9 Quality Evaluation Checkpoints

Quality evaluation ensures that the dataset meets standards of completeness, safety, and clarity before being used in a RAG system. A structured checklist allows students or developers to validate content systematically.

Quality Metrics

Table 9.10 : Quality Evaluation Metrics for RAG Dataset Validation

Metric	Description
Coverage	Ensures all intended FAQ or topic categories are represented fairly.

Redundancy	Checks for duplicate or near-duplicate entries.
Clarity	Confirms that language is readable, consistent, and error-free.
Privacy	Verifies that all sensitive or personal information has been removed.

Quality Audit Form

Learners complete the following form for each dataset they curate:

Table 9.11 : Quality Audit — Checklist

Field	Input / Selection
Dataset Name	_____
Check Date	_____
☐ Public information only	Confirmed
☐ Duplicates removed	Confirmed
☐ Categories balanced	Confirmed
☐ Clean formatting	Confirmed

This audit log helps maintain transparency and accountability in dataset creation.

Cleaning and quality evaluation form the backbone of reliable RAG dataset development. By applying standardized preprocessing rules and conducting thorough quality audits, developers ensure that the resulting corpus is consistent, privacy-safe, and free from bias or redundancy. These steps directly enhance the accuracy and trustworthiness of downstream NLP systems.

9.10 Case Study — Preparing a 1000-Record Retail Customer Support Corpus (Record-Based Approach)

This case study shows how to prepare a 1,000-record dataset for a retail customer-service RAG chatbot using individual text records, not files. Each record represents a single customer-support statement, FAQ entry, or product instruction snippet.

Step-by-Step Implementation Plan (Record-Focused Dataset)

Step 1 — Collect Raw Records

Instead of scraping documents, we gather individual records that represent discrete pieces of information.

Source Type	Example Records
Customer FAQs	“How do I track my order?”
Support responses	“Refunds are processed within 5–7 days.”
Product instructions	“Press and hold the power button for 10 seconds.”
Policy excerpts	“Warranty covers manufacturing defects only.”
Delivery updates	“Your package may arrive later due to weather.”

Goal: Collect 1000 raw records, each 1–5 sentences.

Step 2 — Store All Records in Excel (Record Table)

Each row represents one independent record.

Column	Example Entry
ID	0001
Category	Delivery

Raw_Record	“Your order may be delayed due to weather.”
Source_Type	FAQ
Date_Added	18-02-2025

No files—only structured rows.

Step 3 — Clean & Redact Sensitive Content (Per Record)

Apply transformations to each record:

Before	After
“Contact support at help@retail.com .”	“Contact support at [REDACTED].”
“Order #4589 is delayed.”	“Order [REDACTED] is delayed.”
“cAnCeLLAtIoN PoLiCy”	“Cancellation policy”

Every record becomes standardized and privacy-compliant.

Step 4 — Deduplicate Using Similarity Scores (Record-to-Record Comparison)

Perform pairwise text similarity across records:

Example near-duplicate pair:

Table: Near-Duplicate Detection Example

Item	Text / Value
Record A	“Refunds usually take 5–7 business days.”
Record B	“Refunds are processed within 5 to 7 days.”
Similarity Score	0.91 → Duplicate

Apply the rule:

If similarity $\geq 0.90 \rightarrow$ treat as duplicate

Only keep the best version.

Step 5 — Stratify Records Across Categories

Ensure balanced representation of retail-support topics:

Category	Target Records
Delivery	250
Orders	250
Returns & Refunds	250
Product Support	250
Total	1000 Records

This prevents model bias toward frequent categories like delivery.

Step 6 — Create Dataset Documentation Card (Record Metadata Summary)

A data card describes how the 1000 records were created and cleaned.

Field	Example
Dataset Name	Retail_Records_Corpus_2025
Total Records	1000
Record Types	FAQs, support messages, troubleshooting lines
Categories	Delivery, Orders, Returns, Product Support
Privacy	All identifiers removed
Bias Notes	Dataset currently English-dominant

Step 7 — Chunk Records into RAG-Friendly Passages

If any record exceeds 150–180 words, break it into smaller parts.

However, most retail records are short, so many items will remain single-chunk entries.

Example:

Before Chunking	After Chunking
“If your device does not turn on, ensure it is charged. Press and hold the power button for at least 10 seconds...”	Chunk 1: Troubleshooting – charging requirement; Chunk 2: Troubleshooting – hard reset instructions

Each chunk moves directly into the RAG embedding index.

Final Corpus Structure (Record-Based Dataset Example)

ID	Category	Raw Record	Clean Chunk (RAG-Ready)
0001	Delivery	“Your order may be delayed due to weather conditions.”	“Delivery delay: Order arrival extended due to weather conditions.”
0002	Returns	“Refunds take 5–7 business days to reflect in your account.”	“Refund timeline: 5–7 business days after approval.”
0003	Orders	“Track your package through the order history page.”	“Order tracking: Check progress via order history page.”
0004	Product Support	“If the device doesn't start, check battery level.”	“Troubleshooting: Ensure device battery is charged before restarting.”

Each record becomes a standalone, searchable text asset.

Creating a 1000-record retail support corpus involves gathering individual text records, cleaning and redacting them, removing duplicates, balancing categories, documenting dataset details, and preparing each record as a RAG-ready chunk. This record-centric approach produces a structured, privacy-safe dataset ideal for building customer-support chatbots and retrieval-augmented applications.

9.11 Guided Lab Exercise

This lab activity trains students to curate high-quality datasets suitable for RAG systems. The exercise focuses on cleaning, deduplication, classification, and documentation.

Pair Activity

Students will work in pairs to perform the following real-world tasks:

Task	Description
Clean a 25-record sample dataset	Remove artifacts, standardize text, apply redaction, normalize punctuation.
Remove near duplicates	Use similarity scoring (≥ 0.90 = duplicate).
Classify records into 5 categories	Example: Delivery, Orders, Returns, Product Support, Account Help.
Write a Data Card	Document purpose, source, privacy checks, and bias notes.

Required Submission Components

Submission Item	Purpose
Clean CSV File	Final dataset after cleaning and redaction.
Deduplication Audit Sheet	Shows similarity scores and removed duplicates.
Data Card Document	Provides transparency and accountability.

9.12 Ethical Responsibilities of Data Curators

Data curation carries significant ethical and legal responsibilities. Curators must ensure datasets are safe, lawful, unbiased, and aligned with institutional and organizational standards.

Responsibilities

Responsibility	Requirement
Consent & Legality	Ensure all data is publicly available or used with permission.
No Personal Data Usage	Remove names, phone numbers, emails, IDs, and confidential details.
Proper Source Credit	Acknowledge the origin of datasets and documents.
Regular Audits	Recheck datasets for leakage, bias, and outdated information.

Risks of Negligence

Risk	Consequence
Legal penalties	Violation of data protection laws may result in fines or legal action.
Reputational damage	Institutions or companies may lose trust and credibility.
Academic disqualification	Research may be rejected for ethical violations.

9.13 Module Review Points

A summary of key takeaways from this module:

Concept	Key Insight
Data Quality	Impacts how trustworthy and accurate RAG models are.
Stratified Sampling	Ensures balanced topic representation.
Deduplication	Prevents inflated evaluation performance.
Leakage Prevention	Protects the validity of test metrics.
Data Cards	Improve transparency, traceability, and ethical compliance.

MODULE 10

SHIPPING A DEMO: FROM NOTEBOOK TO INTERACTIVE APPLICATION

10.1 Introduction and Module Overview

Building a generative AI system is incomplete unless its functionality is presented in a usable form. A working demonstration helps validate system design, test user experience, showcase safety controls, and communicate technical methodology clearly. This Module introduces learners to converting notebooks into basic user-facing applications using fully free platforms such as Google Colab Forms, Gradio, and Streamlit (Community tier).

Students also learn how to document assumptions, limitations, safe-use requirements, and disclaimers within their application so that end users understand both what the system can do—and what it must not do. The aim of this Module is deployment education, not large-scale production hosting.

10.2 The Data Lifecycle in RAG Systems

In Retrieval-Augmented Generation (RAG) systems, the data lifecycle refers to the structured flow through which user input is validated, relevant knowledge is retrieved, contextual grounding is applied, and a reliable response is generated with safeguards. This lifecycle ensures that Large Language Models respond using verified external data rather than unsupported assumptions.

Core Stages Explained

A student-facing RAG demo application typically follows a disciplined and transparent flow that balances usability, accuracy, and safety.

Deployment Architecture for Student RAG Demos

Standard Workflow

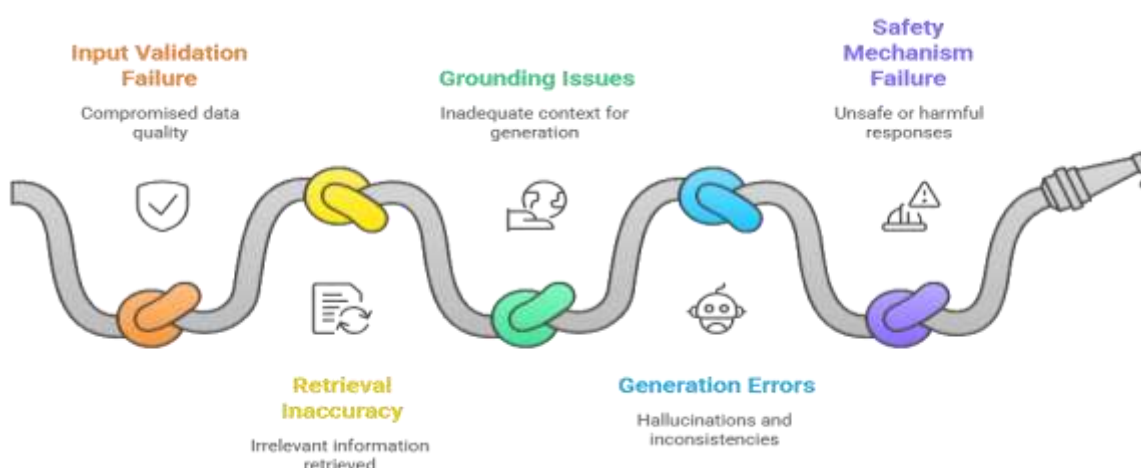


Figure 10.1 : RAG Pipeline Challenges

Table 10.1 : Stage-wise Explanation

Stage	Purpose	Example in Student Demo
User Input	Collects the query from the user	“What is the attendance policy?”
Input Validation	Filters empty or unsafe inputs	Rejects personal or sensitive queries

Retrieve	Fetches relevant records	Policy documents from vector store
Ground	Anchors response to retrieved data	Uses official circular text
Generate	Produces final answer	Natural language response
Answer + Citations	Ensures transparency	Adds source reference
Safety Fallback	Prevents hallucination	“No information available”

This architecture ensures that even beginner-built demos behave responsibly and predictably.

10.3 Deploying via Google Colab Forms

Colab Forms Overview

Google Colab Forms enable quick user interaction within a notebook by converting input cells into simple UI elements. This method is ideal for early-stage demonstrations without external deployment complexity.

Basic Form Example

```
question = input("Ask your campus question:")
```

Capabilities in Educational Context

Colab Forms, though minimal, are sufficient for showcasing RAG concepts in classrooms and evaluations.

Table 10.2 : Use Case Table

Capability	Description
Parameter tuning	Modify prompts or retrieval limits
Demo recording	Screen-record interactive runs
Instructor grading	Easy submission and review
GPU support	Optional acceleration

Table 10.3 : Advantages

Strength	Description
Free & easy	No deployment cost or setup
Notebook-native	Runs inside existing workflow
Classroom-friendly	Ideal for live teaching demos

10.4 Gradio Demo Applications

Introduction to Gradio

Gradio is a Python-based framework that transforms backend functions into interactive web interfaces. It enables students to present AI demos as shareable applications with minimal code.

Sample Gradio Code

```
import gradio as gr

def rag_bot(question):
    return "Answer placeholder with citation."

app = gr.Interface(
```

```

fn = rag_bot,
inputs = "textbox",
outputs = "textbox",
title = "Campus RAG Chatbot"
)

app.launch()

```

Table 10.4 : Key Features

Feature	Benefit
Interactive UI	Improves user experience
Shareable links	Easy demo access
Visual clarity	Better explanation during evaluation

Table 10.5 : Usage Guidelines

Guideline	Reason
No login-based auth	Keeps demo simple
Avoid private datasets	Ensures ethical compliance
Use public or synthetic data	Safe for academic sharing

This module demonstrates how RAG-based NLP systems can be effectively transitioned from experimental notebooks to interactive demos using simple deployment tools. By following a structured data lifecycle, applying transparent retrieval and grounding, and using platforms like Colab Forms or Gradio, students can present reliable, ethical, and evaluation-ready AI applications without heavy infrastructure requirements.

10.5 Streamlit Applications

Streamlit is a Python-based application framework used to build interactive, dashboard-style web applications directly from scripts. In the context of NLP and RAG-based demos, Streamlit enables students to move beyond simple chat interfaces and create rich, visual applications that display model behavior, retrieval quality, and evaluation metrics in a structured manner.

Unlike lightweight demo tools, Streamlit is suitable when the goal is to **demonstrate system performance, transparency, and analysis**, rather than only question–answer interaction.

Basic Streamlit Example

```

import streamlit as st
st.title("Campus Assistant Bot")
user_question = st.text_input("Ask a campus question:")
if user_question:
    st.write("Answer with citation")

```

Explanation

- `st.title()` sets the application heading
 - `st.text_input()` collects user queries
 - Conditional logic displays the model response only when input is provided
- This simple structure can be extended to include retrieval results, citations, and charts.

Table 10.6 Key Capabilities of Streamlit in RAG Demos

Component	Purpose in Demo
Text input	Accepts user questions
Output panels	Displays generated answers
Expanders	Shows retrieved chunks
Charts	Visualizes evaluation scores
Tabs / Pages	Organizes multiple views

Table 10.7 When to Use Streamlit

Use Case	Why Streamlit Fits
Multi-page dashboards	Supports navigation and layout control
Evaluation result display	Easy integration with charts and tables
Chunk ranking visualization	Enables transparency of retrieval step
Instructor review demos	Clear separation of inputs, outputs, and metrics

10.6 Input Validation & Safety Layer

Before any interaction with a Large Language Model, user input must be screened through a dedicated validation and safety layer. This layer acts as a protective boundary that prevents misuse, safeguards privacy, and ensures that the AI system operates strictly within its intended academic scope.

Table 10.8 Input Validation Rules

Rule	Purpose
Token length caps	Prevent context overflow
Keyword blocklist	Block sensitive requests
Regex validation	Prevent injection attempts
PII detection	Protect user privacy

These rules ensure that only clean, relevant, and safe queries reach the RAG pipeline.

Sample Guard Prompt

Instruction to the model:

Reject any input that requests legal advice, medical guidance, financial recommendations, or personal identifiers such as phone numbers, addresses, or identity details.

This guardrail ensures that the model remains informational rather than advisory.

10.7 Error Handling & No-Answer Responses

When to Trigger Fallback

A fallback or no-answer response must be generated under the following conditions:

Table 10.9 Fallback Activation Criteria in Retrieval-Augmented Generation Applications

Condition	Reason
Retrieval similarity below threshold	Insufficient evidence to answer reliably
Conflicting retrieved documents	Risk of misinformation
Banned topic detected	Policy or ethical restriction

Standard No-Answer Message

System Response:

No verified information available.

Please consult official campus administration.

This response maintains user trust by avoiding speculation.

10.8 User Experience (UX) Guidelines

Table 10.10 Design Checklist

UX Element	Rationale
Simple text input	Reduces user friction
Clear safety disclaimers	Sets correct expectations
Citation display	Improves transparency
Neutral language tone	Avoids bias or authority claims
Mobile compatibility	Ensures accessibility

Example Disclaimer Text

This AI assistant provides general campus information only.

Do not use it for personal, medical, legal, or financial decisions.

A robust input validation and safety layer, combined with clear fallback handling and thoughtful UX design, is essential for responsible NLP deployments. These safeguards ensure that student-built AI demos remain ethical, transparent, and trustworthy while delivering reliable information within clearly defined boundaries.

10.9 Documentation Standards

Documentation standards define the minimum written information that must accompany any deployed NLP or RAG-based demo. Proper documentation ensures transparency, reproducibility, ethical compliance, and academic accountability by clearly explaining what the system does, what data it uses, and where its limitations lie.

Well-prepared documentation allows evaluators and users to understand the system without inspecting the code.

Table 10.11 Minimum README Sections

Section	Purpose
Project description	Explains the objective and scope of the demo
Data source summary	Specifies where knowledge is sourced from
RAG pipeline overview	Describes retrieval, grounding, and generation
Safety rules	Lists blocked or restricted topics
Known limitations	Clarifies what the system cannot do
Contact / disclaimer	States institutional responsibility

README Example

Project Title: Campus Assistant Bot

- Data: Public campus handbook FAQs only
- RAG Pipeline: Query → retrieve policy chunks → grounded response
- Safety: Refuses medical, legal, and financial advice
- Limitations: Responses limited to available documents

This concise format communicates key system properties at a glance.

Each deployment must include README documentation.

Minimum README Sections

1. Project description
2. Data source summary
3. RAG pipeline overview
4. Safety rules
5. Known limitations
6. Contact/institution disclaimer

README Example

Campus Assistant Bot

Data: Public handbook FAQs only.

Safety: Refuses medical/financial advice.

Limitations: Responses limited to available documents.

10.10 Capturing Demo Evidence

Demo evidence refers to the artifacts collected to prove that the application works as intended. These records support evaluation, grading, and future reproducibility by visually documenting system behavior and response grounding.

Table 10.12 Required Evidence Types

Evidence	Purpose
Video demo (optional)	Shows live system interaction
UI screenshots	Confirms interface functionality
Query–answer–citation chain	Demonstrates grounding accuracy

Folders

/app → Code & UI

/data → Clean corpus

/eval → Score spreadsheets

/slides → Presentation summary

Clear documentation and well-organized demo evidence are essential for validating NLP deployments in academic settings. By maintaining structured READMEs and verifiable evidence artifacts, students demonstrate not only technical competence but also ethical awareness, transparency, and readiness for real-world AI system evaluation.

10.11 Case Study — Shipping a Campus RAG Chatbot

This case study illustrates the complete lifecycle of deploying a student-built Retrieval-Augmented Generation (RAG) chatbot for campus information. It demonstrates how an experimental notebook-based system is transformed into a usable, safe, and well-documented interactive application suitable for academic evaluation.

Table 10.13 Summary of the Deployment Approach

Step	Description
Notebook testing	RAG components validated using sample queries
UI migration	Core functions wrapped in Gradio interface

Validation layer	Input checks added before model execution
Safety disclaimer	Usage boundaries clearly displayed
Evidence capture	Screenshots and outputs recorded
Documentation	README published with system details

This structured approach ensures both technical correctness and responsible deployment.

Table 10.14 Example User Interaction

Element	Example
Student input	When is the exam registration deadline?
Bot response	The last date for exam registration is August 15.
Citation	Source: ExamNotice2025

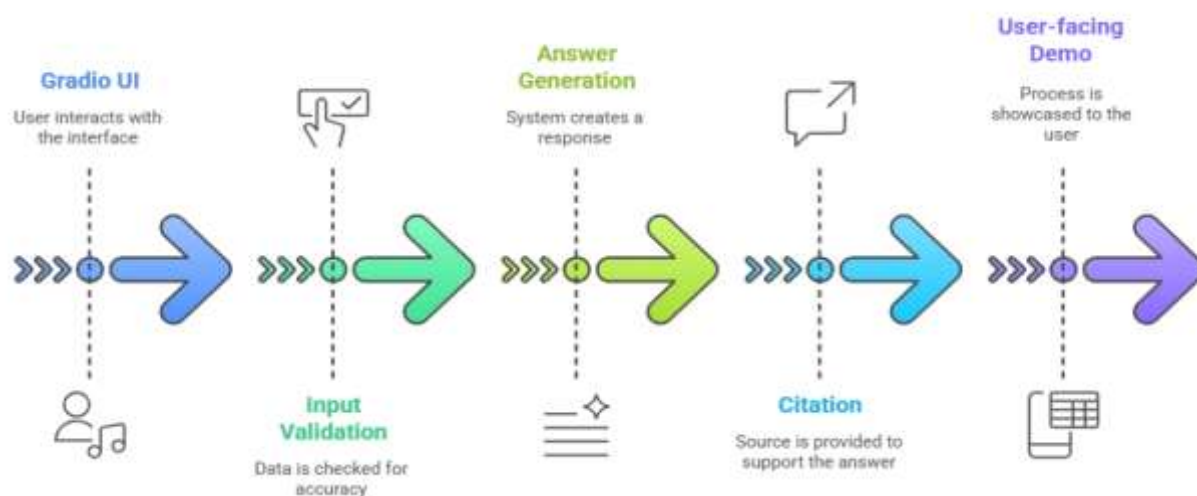


Figure 10.2 : Case Study Flow Diagram

This flow reflects a clean transition from development to deployment.

12. Guided Lab Exercise

The guided lab exercise is a hands-on deployment task designed to help students independently convert a functional RAG script into an interactive demo application while enforcing safety and documentation standards.

Solo Deployment Task

Task	Requirement
UI wrapping	Use Gradio or Streamlit
Disclaimer	Display usage limitations
Safety testing	Verify refusal behavior

Submission Checklist

Item	Status
Working demo link or notebook	Required
Screenshot evidence	Required
README documentation	Required

Completion of all items confirms deployment readiness.

13. Module Review Points

Key Learning	Explanation
Demos bridge build and use	Connects backend logic with user experience
Free hosting tools	Enable cost-free educational deployment
Input validation	Prevents unsafe or unintended usage
Safety disclaimers	Establish ethical boundaries
Documentation	Ensures transparency and compliance

MODULE 11

MINI-PROJECT INTEGRATION, TESTING & REHEARSAL

11.1 Introduction and Module Overview

After deploying a basic interactive demo in Module 10, learners must now consolidate all components into a complete, submission-ready mini-project. This module focuses on integration, validation, rehearsal, and peer review, ensuring that the system works reliably, communicates clearly, and follows responsible AI practices.

The emphasis is not on adding new features, but on stability, clarity, safety compliance, and presentation readiness.

11.2 Learning Objectives

By the end of this module, learners will be able to:

- Integrate data, retrieval, generation, UI, and safety layers into a single package
- Verify fallback behaviour and citation correctness
- Organize project artefacts using a standard folder structure
- Rehearse demo delivery with peer feedback

11.3 Mini-Project Packaging Structure

Students standardize their project layout to improve evaluation and reproducibility.

Table 11.1: Recommended Mini-Project Folder Structure

Folder	Purpose
/data	Clean, approved knowledge corpus
/notebooks	Development and testing notebooks
/app	Gradio or Streamlit application code
/eval	Evaluation logs, similarity scores
/slides	One-slide demo summary

11.4 Validation & Rehearsal Focus Areas

Students explicitly test and rehearse:

Area	Validation Goal
Query handling	Clear input–output flow
Citation display	Evidence-based answers
Fallback triggers	Ethical refusal behaviour
Disclaimer visibility	User awareness

11.5 In-Class Guided Activity

Team Activity

- Finalize demo and one slide
- Conduct peer review using a 3-point checklist:
 1. Is the answer grounded and cited?

2. Does the system refuse unsafe queries?
3. Are limitations clearly communicated?

11.6 Homework

- Freeze the submission package
- Upload draft demo, README, and evidence artefacts

11.7 Micro-Quiz (2 Items)

- Q1 (MCQ): Which folder should store evaluation results?
- Q2 (Scenario-based MCQ): A chatbot answers confidently without citations. What requirement is violated?

Module 11 Summary

This module prepares learners to treat AI demos as complete systems, not experiments. Emphasis on integration, testing, and rehearsal builds confidence and professional readiness.

MODULE 12

ASSESSMENT DAY: EXAMINATION & MINI-PROJECT PRESENTATION

12.1 Introduction and Module Overview

Module 12 is the formal assessment and closure module. Learners demonstrate both conceptual understanding and applied competence through a short MCQ examination and a concise mini-project presentation.

The focus is on clarity, integrity, and responsible communication, rather than system complexity.

12.2 Learning Objectives

By the end of this module, learners will:

- Demonstrate understanding of GenAI, NLP, prompts, and RAG concepts
- Present a mini-project clearly within strict time limits
- Explain system boundaries, limitations, and safety rules

12.3 Assessment Components

Table 12.1: Assessment Structure

Component	Description
MCQ Exam	20 concept-focused questions
Project Presentation	1 slide, 2-minute demo
Q&A	Clarification and reasoning

12.4 Examination Guidelines

- Closed-resource MCQ exam
- Questions test understanding, not memorization
- Integrity statement reinforces responsible AI usage

12.5 Mini-Project Presentation Guidelines

Each team presents:

- Problem statement
- RAG workflow (retrieve → ground → answer)
- Safety and fallback behaviour
- Key limitation

Table 12.2 Evaluation Focus

Criterion	Focus
Technical clarity	Is the logic understandable?
Safety awareness	Are refusals and limits clear?
Communication	Is explanation concise and neutral?

12.6 In-Class Tasks (10–15 Minutes)

- Sit for MCQ exam
- Deliver 2-minute team presentation

12.7 Homework (Optional)

- Post-course reflection on:
 - What worked
 - What failed
 - What would be improved in real deployment

12.8 Micro-Quiz (2 Items)

- Q1 (MCQ): What is the primary purpose of retrieval in RAG?
- Q2 (Scenario-based MCQ): A system refuses a valid query. What component should be reviewed?

Module 12 Summary

Assessment Day reinforces that effective GenAI practitioners must balance technical skill, ethical judgment, and communication clarity. The ability to explain system limits is valued as highly as building the system itself.